

Module No. 5	Transport Layer
--------------	-----------------

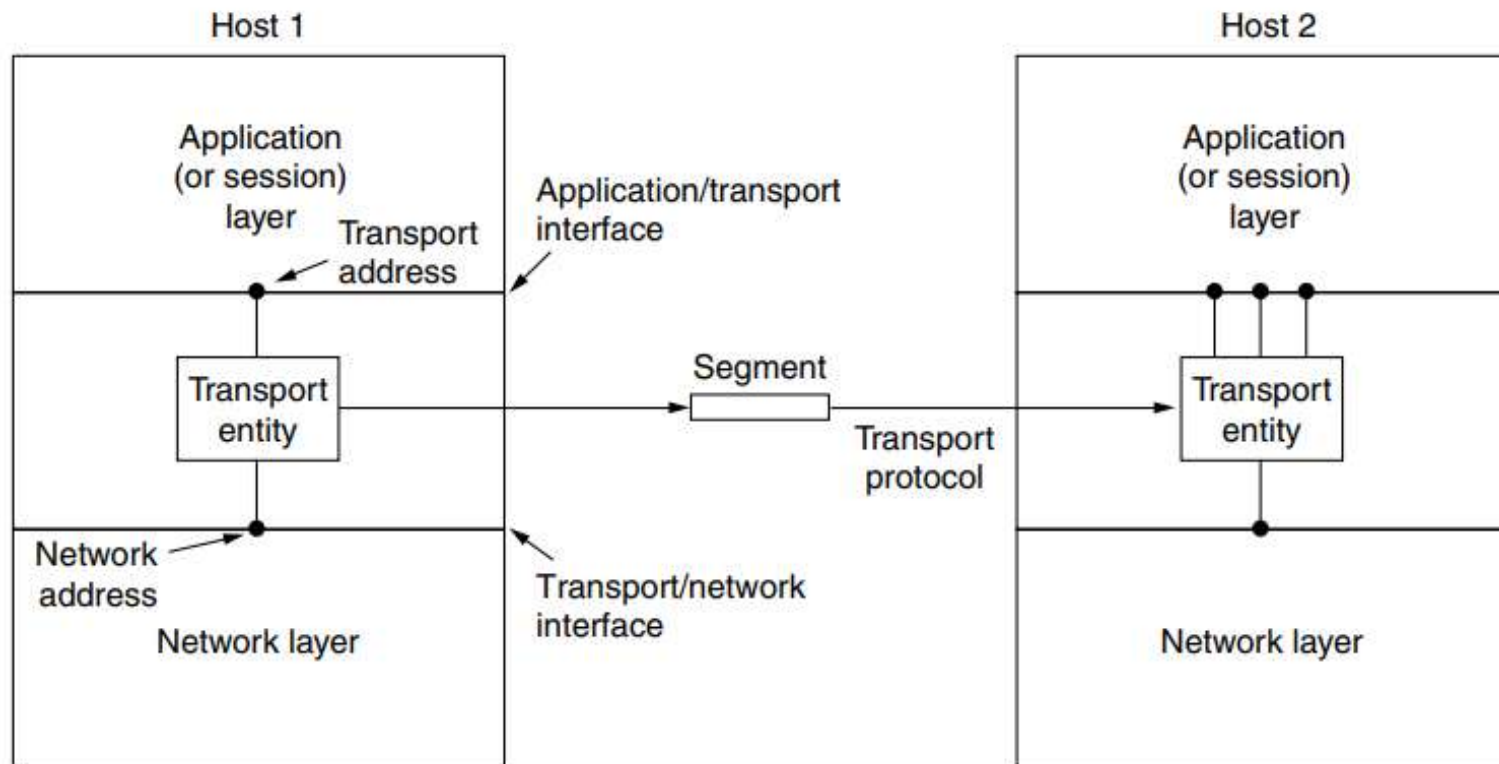
Connection Oriented and connection less service, TCP and UDP, Port Addressing, Remote Procedure Call, Flow Control vs Congestion Control, Quality of Service.

Services provided by the Transport Layer

- Services provided by the UDP and TCP combinedly

- The ultimate goal of the transport layer is to provide efficient, reliable, and cost-effective data transmission service to its users, normally processes in the application layer.
- The software and/or hardware within the transport layer that does the work is called the **transport entity**.
- The transport entity can be located in the operating system kernel, in a library package bound into network applications, in a separate user process, or even on the network interface card.

The network, transport, and application layers.

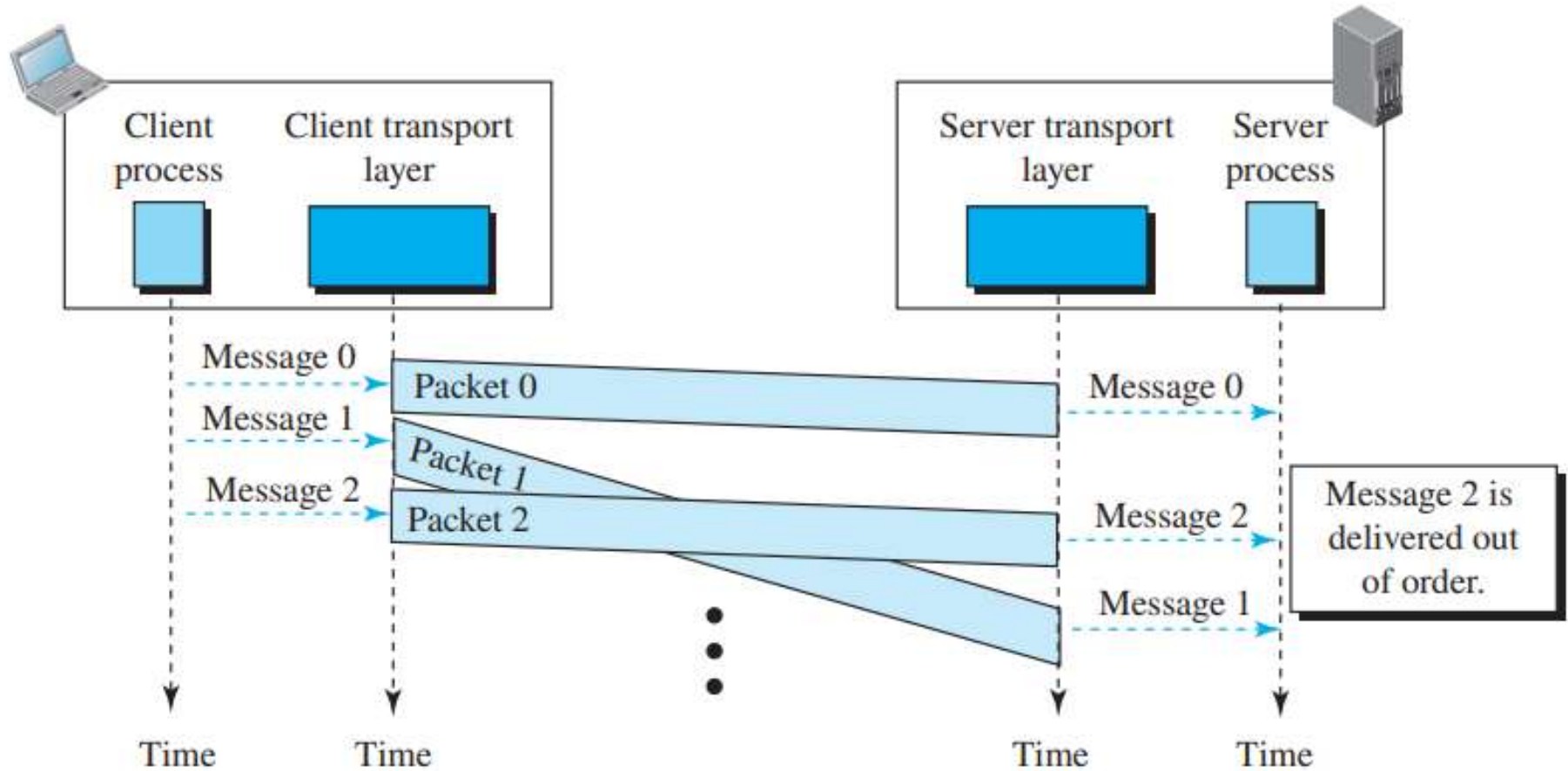


There are two types of Connection services provided by transport layer.

- Connection-oriented
- Connectionless

Transport Protocol Data Unit (TPDU) : Transport data unit i.e., Header + Data unit

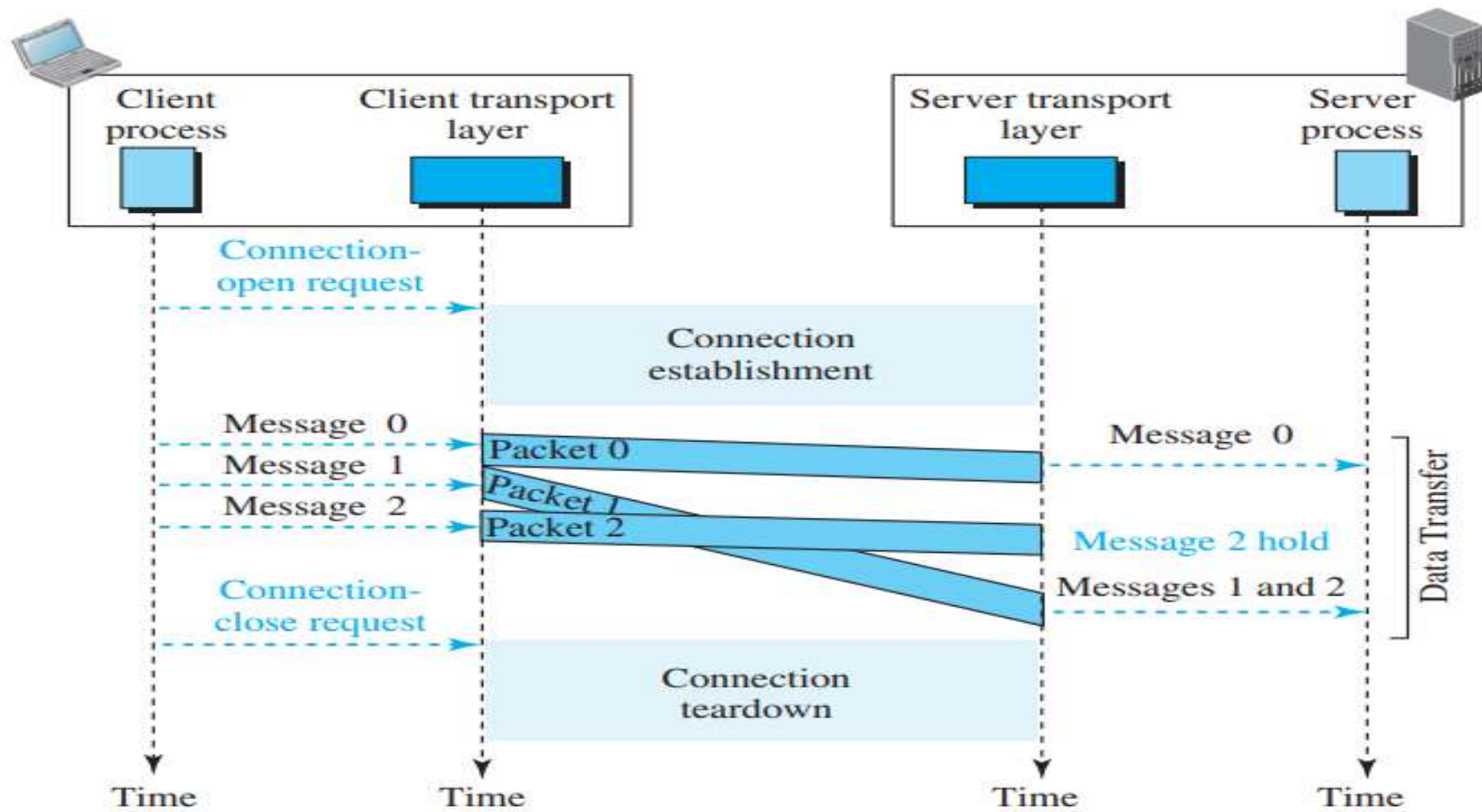
Connectionless service



In general, connections have three phases:

- Establishment
- Data transfer
- Release

Connection-oriented service



- Some primitives that gives the essential understanding of what a connection-oriented transport interface has to do are given below.
- It allows application programs to establish, use, and then release connections, which is sufficient for many applications.

The primitives for a simple transport service.

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	Request a release of the connection

Simple Connection steps

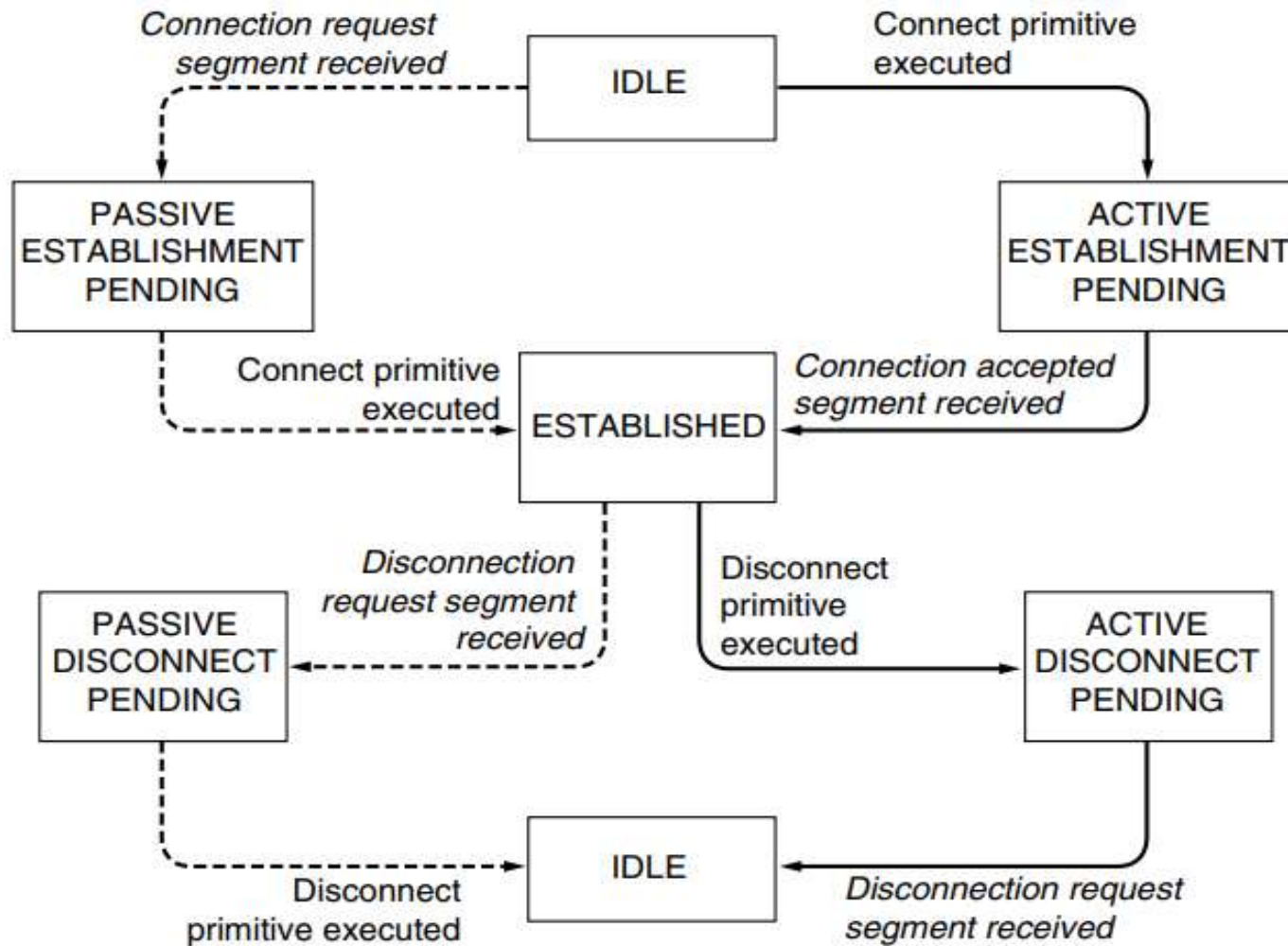
1. To start with, the server executes a LISTEN primitive, typically by calling a library procedure that makes a system call that blocks the server until a client turns up.
2. When a client wants to talk to the server, it executes a CONNECT primitive.
3. The transport entity carries out this primitive by blocking the caller and sending a packet to the server.
4. Encapsulated in the payload of this packet is a transport layer message for the server's transport entity.
5. The client's CONNECT call causes a CONNECTION REQUEST segment to be sent to the server.

6. When it arrives, the transport entity checks to see that the server is blocked on a LISTEN (i.e., is interested in handling requests).
7. If so, it then unblocks the server and sends a CONNECTION ACCEPTED segment back to the client. When this segment arrives, the client is unblocked and the connection is established.
8. Data can now be exchanged using the SEND and RECEIVE primitives.
 - In the simplest form, either party can do a (blocking) RECEIVE to wait for the other party to do a SEND. When the segment arrives, the receiver is unblocked. It can then process the segment and send a reply.
9. When a connection is no longer needed, it must be released to free up table space within the two transport entities. For this the user will issue a DISCONNECT primitive.

Disconnection has two variants:

- ✓ Asymmetric
 - ✓ Symmetric
-
- In the **Asymmetric variant**, either transport user can issue a DISCONNECT primitive, which results in a DISCONNECT segment being sent to the remote transport entity. Upon its arrival, the connection is released.
 - In the **Symmetric variant**, each direction is closed separately, independently of the other one. When one side does a DISCONNECT, that means it has no more data to send but it is still willing to accept data from its partner. In this model, a connection is released when both sides have done a DISCONNECT.

A state diagram for a simple connection management scheme.



- ❖ Transitions labelled in italics are caused by packet arrivals.
- ❖ The solid lines show the client's state sequence.
- ❖ The dashed lines show the server's state sequence.

Port Addressing

- Operating Systems today support both multiuser and multiprogramming environments.
- A remote computer can run several server programs at the same time, just as several local computers can run one or more client programs at the same time.
- For communication, we must define the local host, local process, remote host, and remote process.
- The local host and the remote host are defined using **IP addresses**.
- To define the processes, second identifiers called **Port Numbers** are needed.
- In the TCP/IP protocol suite, the port numbers are integers between 0 and 65,535 (16 bits).

There are two types of port numbers exist.

- ✓ Ephemeral port number
- ✓ Well-known port number

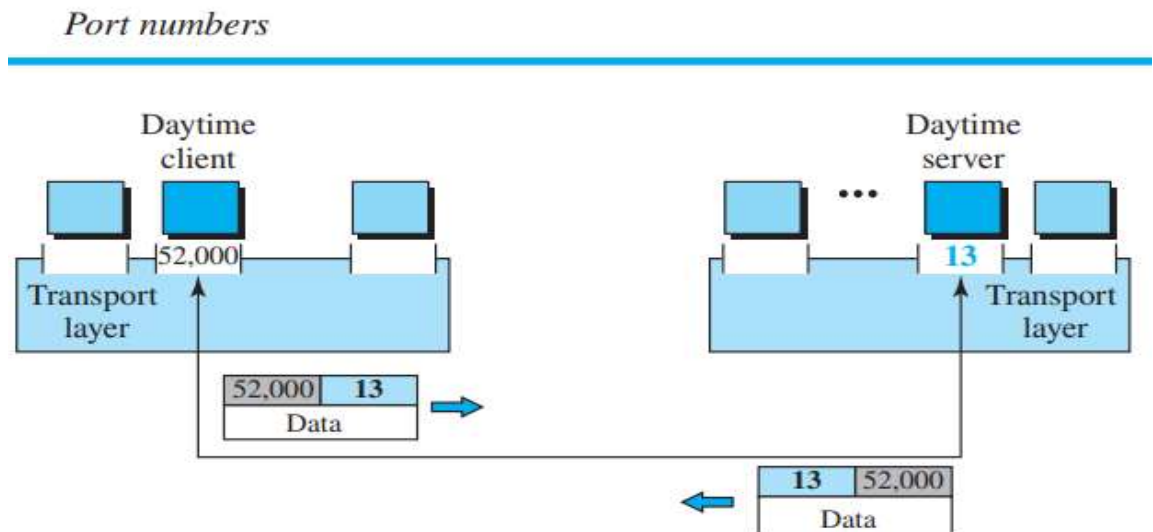
Ephemeral port number

- The word **ephemeral** means “*short-lived*” and is used because the life of a client is normally short.
- The client program defines itself with this type of port number.
- An ephemeral port number is ***recommended to be greater than 1023*** for some client/server programs to work properly.

Well-known port number

- The server process must also define itself with a port number and this cannot be chosen randomly.
 - If the computer at the server site runs a server process and assigns a random number as the port number, the process at the client site that wants to access that server and use its services will not know the port number.
- TCP/IP has decided to use universal port numbers for servers called **Well-known port numbers**.
- Every client process knows the well-known port number of the corresponding server process.

For example, while the daytime client process, an application program, can use an **ephemeral port number**, 52,000, to identify itself, the daytime server process must use the **well-known port number** 13.



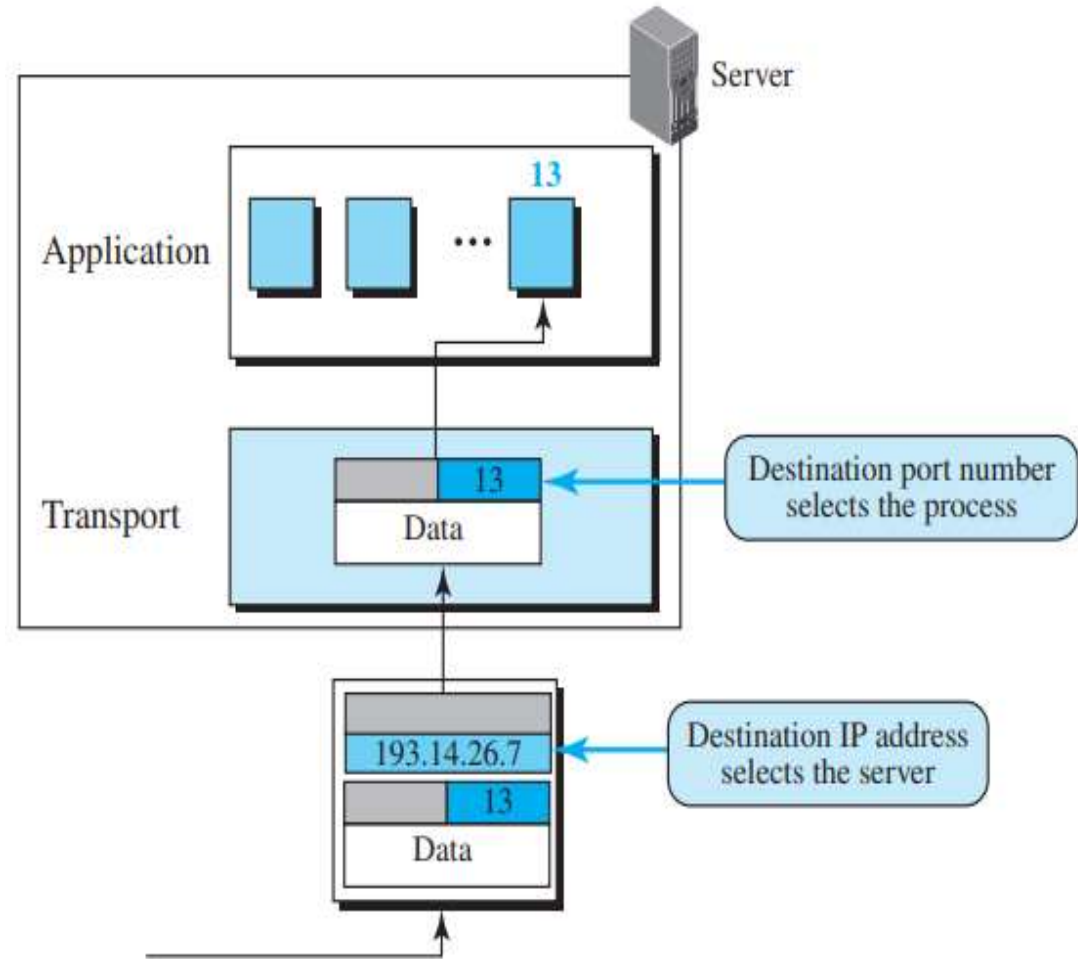
Some well-known ports used with three transport protocols

<i>Port</i>	<i>Protocol</i>	<i>UDP</i>	<i>TCP</i>	<i>SCTP</i>	<i>Description</i>
7	Echo	✓	✓	✓	Echoes back a received datagram
9	Discard	✓	✓	✓	Discards any datagram that is received
11	Users	✓	✓	✓	Active users
13	Daytime	✓	✓	✓	Returns the date and the time
17	Quote	✓	✓	✓	Returns a quote of the day
19	Chargen	✓	✓	✓	Returns a string of characters
20	FTP-data		✓	✓	File Transfer Protocol

<i>Port</i>	<i>Protocol</i>	<i>UDP</i>	<i>TCP</i>	<i>SCTP</i>	<i>Description</i>
21	FTP-21		✓	✓	File Transfer Protocol
23	TELNET		✓	✓	Terminal Network
25	SMTP		✓	✓	Simple Mail Transfer Protocol
53	DNS	✓	✓	✓	Domain Name System
67	DHCP	✓	✓	✓	Dynamic Host Configuration Protocol
69	TFTP	✓	✓	✓	Trivial File Transfer Protocol
80	HTTP		✓	✓	Hypertext Transfer Protocol
111	RPC	✓	✓	✓	Remote Procedure Call
123	NTP	✓	✓	✓	Network Time Protocol
161	SNMP-server	✓			Simple Network Management Protocol
162	SNMP-client	✓			Simple Network Management Protocol

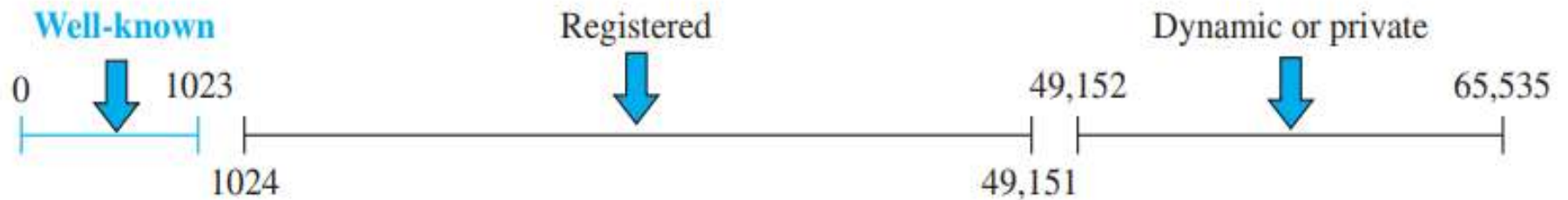
IP addresses versus port numbers

- The IP addresses and port numbers play different roles in selecting the final destination of data.
- The destination IP address defines the host among the different hosts in the world.
- After the host has been selected, the port number defines one of the processes on this particular host.



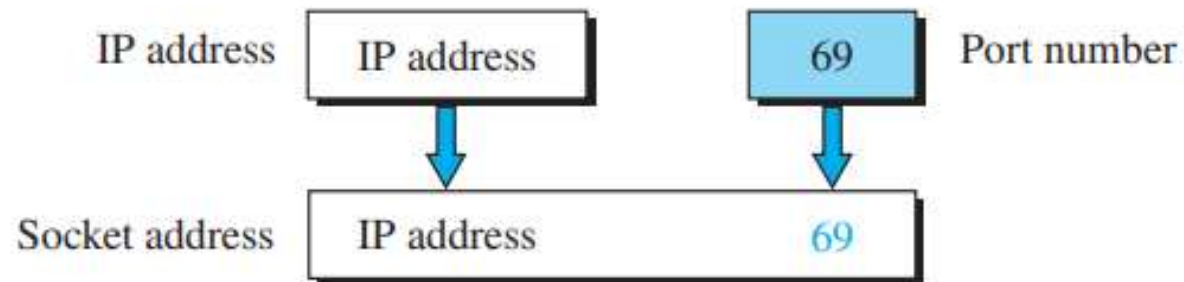
Port number ranges

- The Internet Corporation for Assigned Names and Numbers (ICANN) has divided the port numbers into three ranges.
 - ✓ **Well-known ports:** The ports ranging from 0 to 1023 are assigned and controlled by ICANN.
 - ✓ **Registered ports:** The ports ranging from 1024 to 49,151 are not assigned or controlled by ICANN. They can only be registered with ICANN to prevent duplication.
 - ✓ **Dynamic ports:** The ports ranging from 49,152 to 65,535 are neither controlled nor socket addresses.



Socket address

- A transport-layer protocol in the TCP suite needs both the IP address and the port number, at each end, to make a connection.
- The combination of an IP address and a port number is called a **Socket address**.
- The client socket address defines the client process uniquely, just as the server socket address defines the server process uniquely.



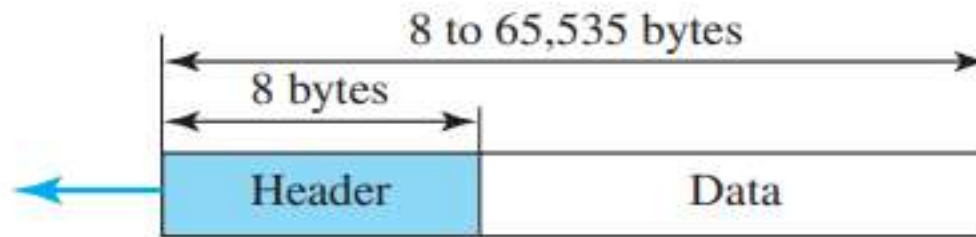
UDP (User Datagram Protocol)

- UDP provides a way for applications to send encapsulated IP datagrams without having to establish a connection.
- UDP is described in RFC 768.
- UDP does not do flow control, congestion control, or retransmission upon receipt of a bad segment. All of that is up to the user processes.

RFC (Request for Comments) is a formal document published by the Internet Engineering Task Force (IETF) that describes technical specifications, standards, and best practices related to internet and networking technologies. These

- UDP transmits segments consisting of an 8-byte header followed by the payload.
- The **two ports** serve to identify the endpoints within the source and destination machines.
 - When a UDP packet arrives, its payload is handed to the process attached to the destination port.
 - This attachment occurs when the BIND primitive is used.
 - In fact, the main value of UDP over just using raw IP is the addition of the **source and destination ports**.
 - Without the port fields, the transport layer would not know what to do with each incoming packet.
 - With them, it delivers the embedded segment to the correct application.

User datagram packet format



a. UDP user datagram



b. Header format

- The **source port** is primarily needed when a reply must be sent back to the source.
- By copying the **Source port field** from the incoming segment into the **Destination port field** of the outgoing segment, the process sending the reply can specify which process on the sending machine is to get it.

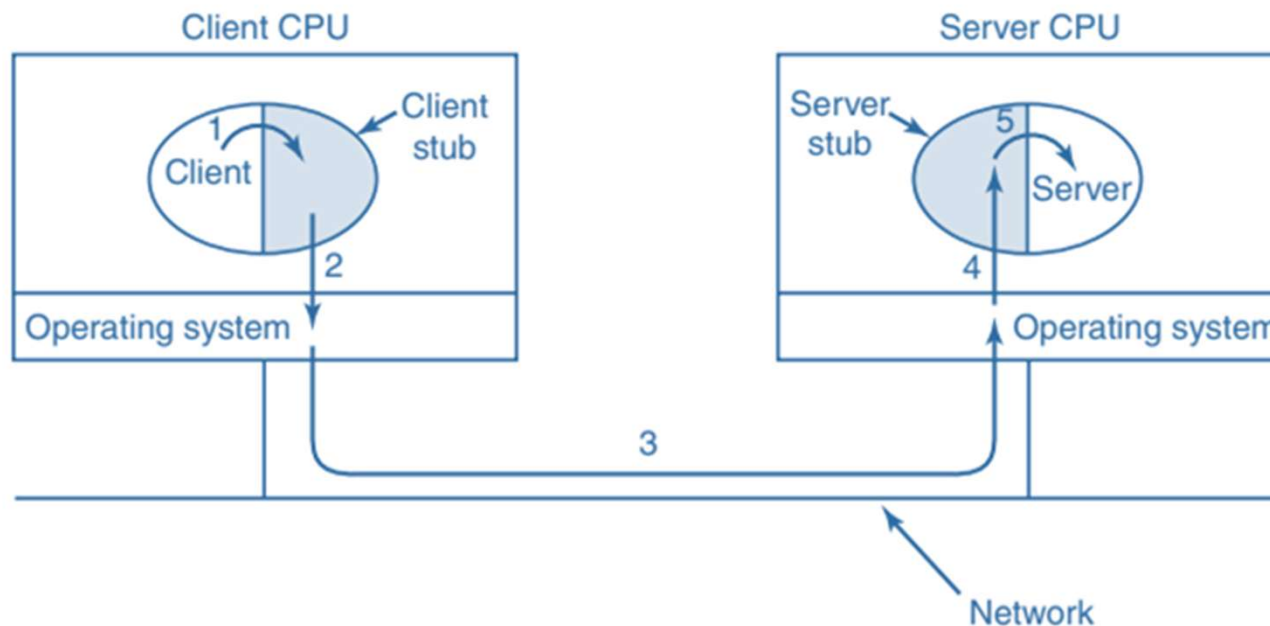
- The **UDP length field** includes the 8-byte header and the data.
 - ✓ The minimum length is 8 bytes, to cover the header.
 - ✓ The maximum length is 65,515 bytes, which is lower than the largest number that will fit in 16 bits because of the size limit on IP packets.
- An optional **Checksum** is also provided for extra reliability.
 - ✓ It checksums the header, the data, and a conceptual IP pseudoheader.
 - ✓ When performing this computation, the Checksum field is set to zero and the data field is padded out with an additional zero byte if its length is an odd number. The checksum algorithm is simply to add up all the 16-bit words in one's complement and to take the one's complement of the sum. As a consequence, when the receiver performs the calculation on the entire segment, including the Checksum field, the result should be 0.

Remote Procedure Call (RPC)

- When a process on machine 1 calls a procedure on machine 2, the calling process on 1 is suspended and execution of the called procedure takes place on 2.
- Information can be transported from the **Caller** to the **Callee** in the parameters and can come back in the procedure result.
- No message passing is visible to the application programmer.
- This technique is known as **RPC (Remote Procedure Call)** and has become the basis for many networking applications.
- Traditionally, the calling procedure is known as the **Client** and the called procedure is known as the **Server**.

- In the simplest form, to call a remote procedure, the client program must be bound with a small library procedure, called the **client stub**, that represents the server procedure in the client's address space.
- Similarly, the server is bound with a procedure called the **server stub**.
- These procedures hide the fact that the procedure call from the client to the server is not local.

Steps in making a remote procedure call. The stubs are shaded.



- Step 1 is the client calling the **client stub**.
 - ✓ This call is a local procedure call, with the parameters pushed onto the stack in the normal way.
- Step 2 is the client stub packing the parameters into a message and making a system call to send the message.
 - ✓ Packing the parameters is called **Marshaling**.
- Step 3 is the operating system sending the message from the client machine to the server machine.
- Step 4 is the operating system passing the incoming packet to the **server stub**.
- Finally, step 5 is the server stub calling the server procedure with the **Unmarshaled parameters**.
The reply traces the same path in the other direction.

The following is the contents of a UDP header in hexadecimal format.

CB84000D001C001C

- a. What is the source port number?
- b. What is the destination port number?
- c. What is the total length of the user datagram?
- d. What is the length of the data?
- e. Is the packet directed from a client to a server or vice versa?
- f. What is the client process?

Solution

- a. The source port number is the first four hexadecimal digits $(CB84)_{16}$, which means that the source port number is 52,100.
- b. The destination port number is the second four hexadecimal digits $(000D)_{16}$, which means that the destination port number is 13.
- c. The third four hexadecimal digits $(001C)_{16}$ define the length of the whole UDP packet as 28 bytes.
- d. The length of the data is the length of the whole packet minus the length of the header, or $28 - 8 = 20$ bytes.
- e. Because the destination port number is 13 (well-known port), the packet is from the client to the server.
- f. The client process is the Daytime

Assume we are using a real-time interactive application, such as Skype. Audio and video are divided into frames and sent one after another. If the transport layer is supposed to resend a corrupted or lost frame, the synchronizing of the whole transmission may be lost. The viewer suddenly sees a blank screen and needs to wait until the second transmission arrives. This is not tolerable. However, if each small part of the screen is sent using one single user datagram, the receiving UDP can easily ignore the corrupted or lost packet and deliver the rest to the application program. That part of the screen is blank for a very short period of time, which most viewers do not even notice.

UDP Services

Process-to-Process Communication: UDP provides process-to-process communication using socket addresses, a combination of IP addresses and port numbers.

Connectionless Services: UDP provides a connectionless service. Each user datagram sent by UDP is an independent datagram. The user datagrams are not numbered. Also, unlike TCP, there is no connection establishment and no connection termination. This means that each user datagram can travel on a different path.

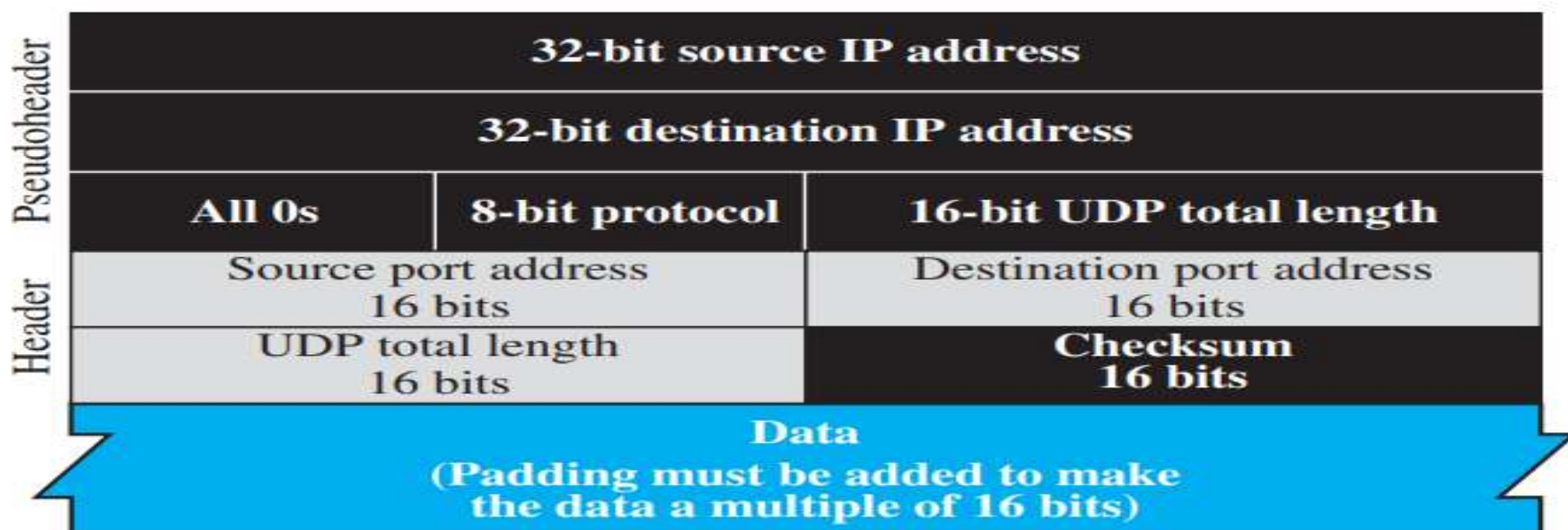
Only those processes sending short messages, messages less than 65,507 bytes (65,535 minus 8 bytes for the UDP header and minus 20 bytes for the IP header), can use UDP.

Flow Control: UDP is a very simple protocol. There is no flow control, and hence no window mechanism. The receiver may overflow with incoming messages.

Error Control: There is no error control mechanism in UDP except for the checksum. This means that the sender does not know if a message has been lost or duplicated. When the receiver detects an error through the checksum, the user datagram is silently discarded. The lack of error control means that the process using UDP should provide for this service, if needed.

Checksum: UDP checksum calculation includes [three sections](#): a [pseudoheader](#), the [UDP header](#), and the [data coming from the application layer](#). The pseudoheader is the part of the header of the IP packet in which the user datagram is to be encapsulated with some fields filled with 0s. The value of the protocol field for UDP is 17. If this value is changed during transmission, the checksum calculation at the receiver will detect it and UDP drops the packet. It is not delivered to the wrong protocol.

Pseudoheader for checksum calculation



What value is sent for the checksum in each one of the following hypothetical situations?

- a. The sender decides not to include the checksum.
- b. The sender decides to include the checksum, but the value of the sum is all 1s.
- c. The sender decides to include the checksum, but the value of the sum is all 0s.

Solution

- a. The value sent for the checksum field is all 0s to show that the checksum is not calculated.
- b. When the sender complements the sum, the result is all 0s; the sender complements the result again before sending. The value sent for the checksum is all 1s. The second complement operation is needed to avoid confusion with the case in part a.
- c. This situation never happens because it implies that the value of every term included in the calculation of the sum is all 0s, which is impossible; some fields in the pseudoheader have nonzero values.

Congestion Control : Since UDP is a connectionless protocol, it does not provide congestion control. UDP assumes that the packets sent are small and sporadic and cannot create congestion in the network. This assumption may or may not be true today, when UDP is used for interactive real-time transfer of audio and video.

Encapsulation and Decapsulation : To send a message from one process to another, the UDP protocol encapsulates and decapsulates messages.

Queuing : In UDP, queues are associated with ports. At the client site, when a process starts, it requests a port number from the operating system. Some implementations create both an incoming and an outgoing queue associated with each process. Other implementations create only an incoming queue associated with each process.

Multiplexing and Demultiplexing : In a host running a TCP/IP protocol suite, there is only one UDP but possibly several processes that may want to use the services of UDP. To handle this situation, UDP multiplexes and demultiplexes.

Comparison between UDP and Generic Simple Protocol: The only difference is that UDP provides an optional checksum to detect corrupted packets at the receiver site. If the checksum is added to the packet, the receiving UDP can check the packet and discard the packet if it is corrupted. No feedback, however, is sent to the sender.

Examples

- A client-server application such as DNS uses the services of UDP because a client needs to send a short request to a server and to receive a quick response from it. The request and response can each fit in one user datagram. Since only one message is exchanged in each direction, the connectionless feature is not an issue; the client or server does not worry that messages are delivered out of order.
- A client-server application such as SMTP, which is used in electronic mail, cannot use the services of UDP because a user might send a long e-mail message, which could include multimedia (images, audio, or video). If the application uses UDP and the message does not fit in one user datagram, the message must be split by the application into different user datagrams. Here the connectionless service may create problems. The user datagrams may arrive and be delivered to the receiver application out of order. The receiver application may not be able to reorder the pieces. This means the connectionless service has a disadvantage for an application program that sends long messages. In SMTP, when we send a message, we do not expect to receive a response quickly (sometimes no response is required). This means that the extra delay inherent in connection-oriented service is not crucial for SMTP.

Typical Applications

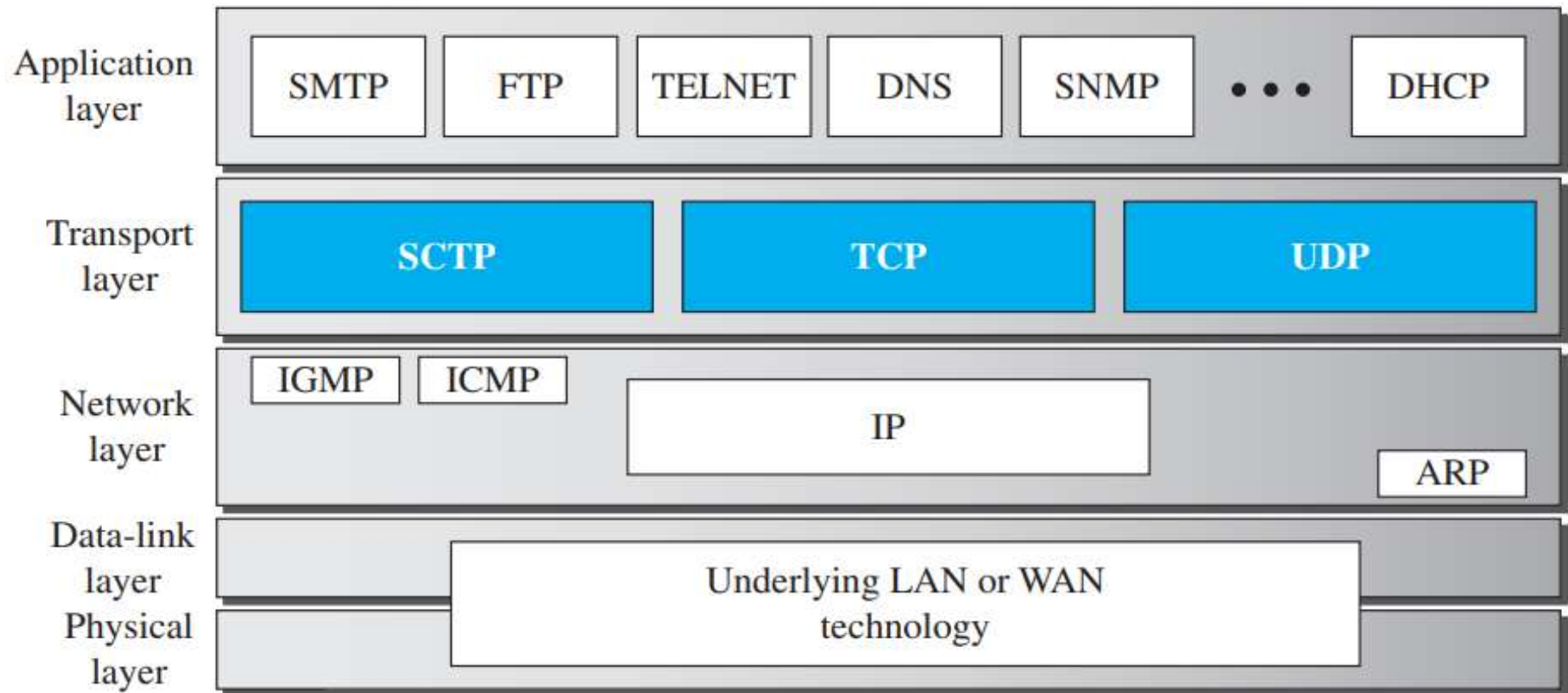
The following shows some typical applications that can benefit more from the services of UDP than from those of TCP.

- UDP is suitable for a process that requires simple request-response communication with little concern for flow and error control.
- It is **not** usually used for a process such as FTP that needs to send bulk data.
- UDP is a suitable transport protocol for multicasting. Multicasting capability is embedded in the UDP software but not in the TCP software.
- UDP is used for management processes such as SNMP.
- UDP is used for some route updating protocols such as Routing Information Protocol (RIP).
- UDP is normally used for interactive real-time applications that cannot tolerate uneven delay between sections of a received message.

Transmission Control Protocol (TCP)

- A connection-oriented, reliable protocol.
- TCP explicitly defines connection establishment, data transfer, and connection teardown phases to provide a connection-oriented service.
- TCP uses a combination of the Go-Back-N (GBN) and Selective-Repeat (SR) protocols to provide reliability.
- To achieve this goal, TCP uses checksum (for error detection), retransmission of lost or corrupted packets, cumulative and selective acknowledgments, and timers.

Position of transport-layer protocols in the TCP/IP protocol suite



TCP Services

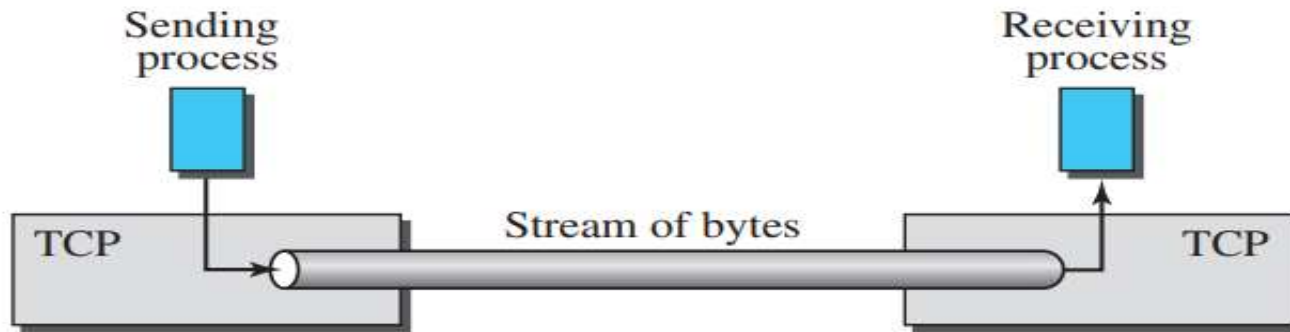
Process-to-Process Communication: As with UDP, TCP provides process-to-process communication using port numbers.

Stream Delivery Service: TCP, unlike UDP, is a stream-oriented protocol.

In UDP, a process sends messages with predefined boundaries to UDP for delivery. UDP adds its own header to each of these messages and delivers it to IP for transmission. **Each message from the process is called a user datagram**, and becomes, eventually, one IP datagram. Neither IP nor UDP recognizes any relationship between the datagrams.

TCP, on the other hand, allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes. TCP creates an environment in which the two processes seem to be connected by an imaginary “tube” that carries their bytes across the Internet.

Stream delivery



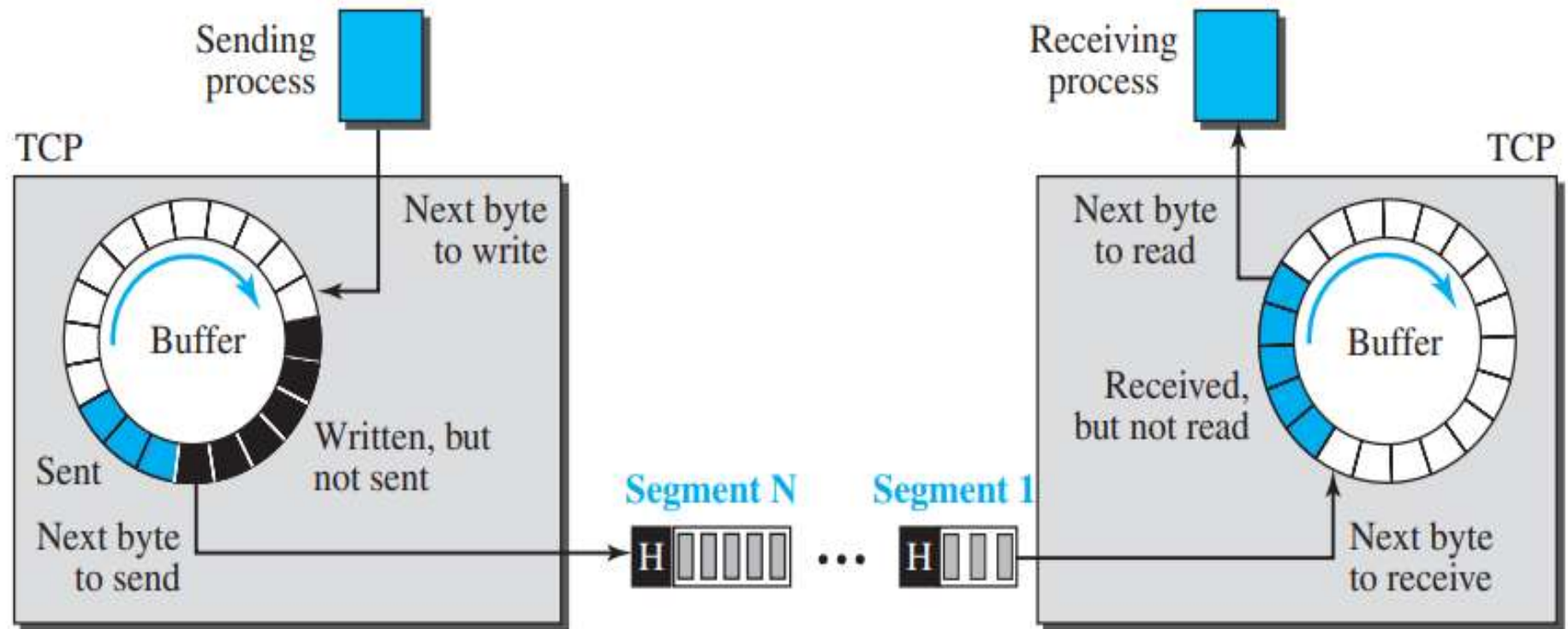
Sending and Receiving Buffers: Because the sending and the receiving processes may not necessarily write or read data at the same rate, TCP needs buffers for storage. There are two buffers, the sending buffer and the receiving buffer, one for each direction. One way to implement a buffer is to use a circular array of 1-byte locations.

At the sender, the buffer has three types of chambers. The white section contains empty chambers that can be filled by the sending process (producer). The colored area holds bytes that have been sent but not yet acknowledged. The TCP sender keeps these bytes in the buffer until it receives an acknowledgment. The shaded area contains bytes to be sent by the sending TCP. TCP may be able to send only part of this shaded section. This could be due to the slowness of the receiving process or to congestion in the network. Also note that, after the bytes in the colored chambers are acknowledged, the chambers are recycled and available for use by the sending process.

The operation of the buffer at the receiver is simpler. **The circular buffer is divided into two areas.** The white area contains empty chambers to be filled by bytes received from the network. The coloured sections contain received bytes that can be read by the receiving process. When a byte is read by the receiving process, the chamber is recycled and added to the pool of empty chambers.

Segments: The network layer, as a service provider for TCP, needs to send data in packets, not as a stream of bytes. **At the transport layer, TCP groups a number of bytes together into a packet called a segment.** TCP adds a header to each segment (for control purposes) and delivers the segment to the network layer for transmission. The segments are encapsulated in an IP datagram and transmitted. This entire operation is transparent to the receiving process.

TCP segments



Full-Duplex Communication

TCP offers *full-duplex service*, where data can flow in both directions at the same time. Each TCP endpoint then has its own sending and receiving buffer, and segments move in both directions.

Multiplexing and Demultiplexing

Like UDP, TCP performs multiplexing at the sender and demultiplexing at the receiver. However, since TCP is a connection-oriented protocol, a connection needs to be established for each pair of processes.

Connection-Oriented Service

TCP, unlike UDP, is a connection-oriented protocol. When a process at site A wants to send to and receive data from another process at site B, the following three phases occur:

1. The two TCP's establish a logical connection between them.
2. Data are exchanged in both directions.
3. The connection is terminated.

Note that this is a logical connection, not a physical connection. The TCP segment is encapsulated in an IP datagram and can be sent out of order, or lost or corrupted, and then resent. Each may be routed over a different path to reach the destination. There is no physical connection. TCP creates a stream-oriented environment in which it accepts the responsibility of delivering the bytes in order to the other site.

Reliable Service

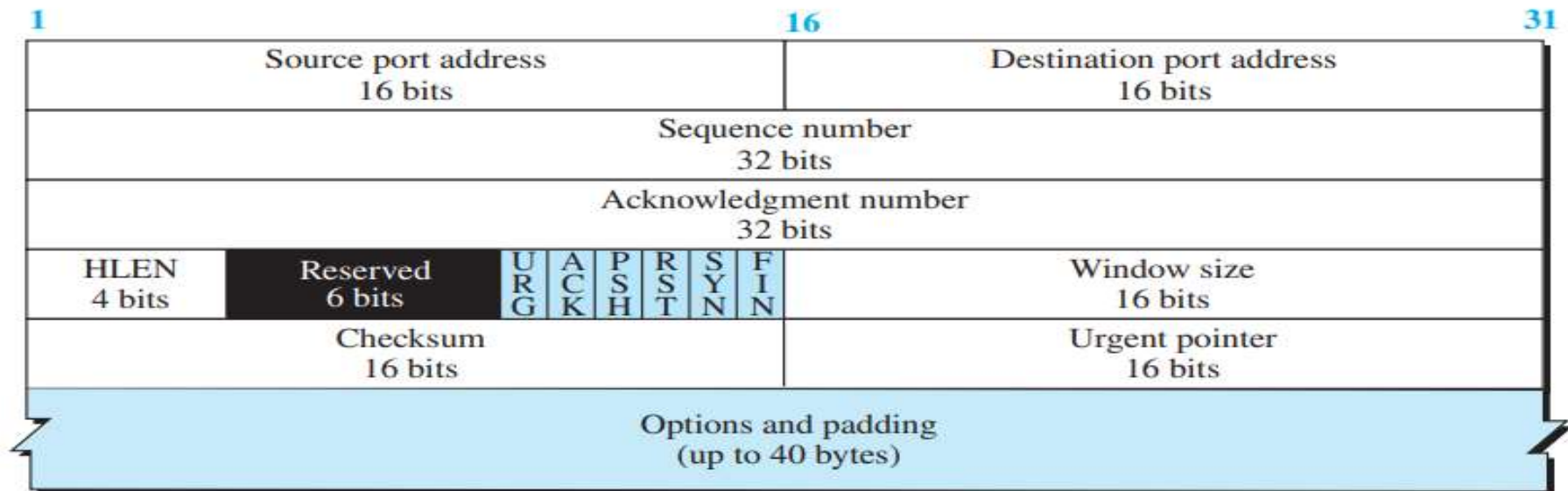
TCP is a reliable transport protocol. It uses an acknowledgment mechanism to check the safe and sound arrival of data.

TCP segment format

- A packet in TCP is called a **segment**. The segment consists of a header of 20 to 60 bytes, followed by data from the application program. The header is 20 bytes if there are no options and up to 60 bytes if it contains options.



a. Segment



b. Header

Source port address

- A 16-bit field that defines the port number of the application program in the host that is sending the segment.

Destination port address

- A 16-bit field that defines the port number of the application program in the host that is receiving the segment.

Sequence number

- This 32-bit field defines the number assigned to the first byte of data contained in this segment.
- To ensure connectivity, each byte to be transmitted is numbered.
- The sequence number tells the destination which byte in this sequence is the first byte in the segment. During connection establishment each party uses a random number generator to create an **Initial Sequence Number (ISN)**, which is usually different in each direction.

Acknowledgment number

- This 32-bit field defines the byte number that the receiver of the segment is expecting to receive from the other party.
- If the receiver of the segment has successfully received byte number **x** from the other party, it returns **x + 1** as the acknowledgment number.
- Acknowledgment and data can be piggybacked together.

Header length

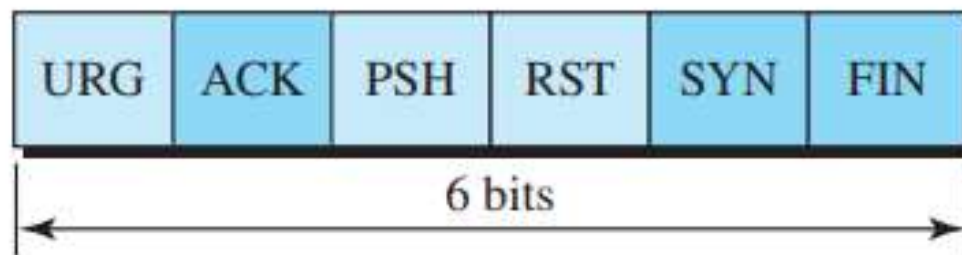
- This 4-bit field indicates the number of **4-byte words** in the TCP header.
- The length of the header can be between 20 and 60 bytes.
- Therefore, the value of this field is always between

5 ($5 \times 4 = 20$) and **15** ($15 \times 4 = 60$) i.e., **5 4-byte words**

Control

- This field defines six different control bits or flags.
- One or more of these bits can be set at a time.
- These bits enable flow control, connection establishment and termination, connection abortion, and the mode of data transfer in TCP.

Control field



URG: Urgent pointer is valid

ACK: Acknowledgment is valid

PSH: Request for push

RST: Reset the connection

SYN: Synchronize sequence numbers

FIN: Terminate the connection

Window size

- This field defines the window size of the sending TCP in bytes.
- The length of this field is 16 bits, which means that the maximum size of the window is 65,535 bytes. This value is normally referred to as the receiving window (rwnd) and is determined by the receiver.
- The sender must obey the dictation of the receiver in this case.

Checksum

- This 16-bit field contains the checksum.
- The calculation of the checksum for TCP follows the same procedure as the one described for UDP.
- The use of the checksum in the UDP datagram is optional, whereas the use of the checksum for TCP is mandatory.

Urgent pointer

- This 16-bit field, which is valid only if the urgent flag is set, is used when the segment contains urgent data.
- It defines a value that must be added to the sequence number to obtain the number of the last urgent byte in the data section of the segment.

Options

- There can be up to 40 bytes of optional information in the TCP header.

0C1F 0050 0000 0001 0000 001C 5010 01F6 E1D4 0000

- a. What is the source port number in decimal?
- b. What is the destination port number in decimal?

0C1F (Source Port)

0050 (Destination Port)

0000 0001 (Sequence Number)

0000 001C (Acknowledgment Number)

5010 (Header Length + Flags)

01F6 (Window Size)

E1D4 (Checksum)

0000 (Urgent Pointer)

TCP Connection

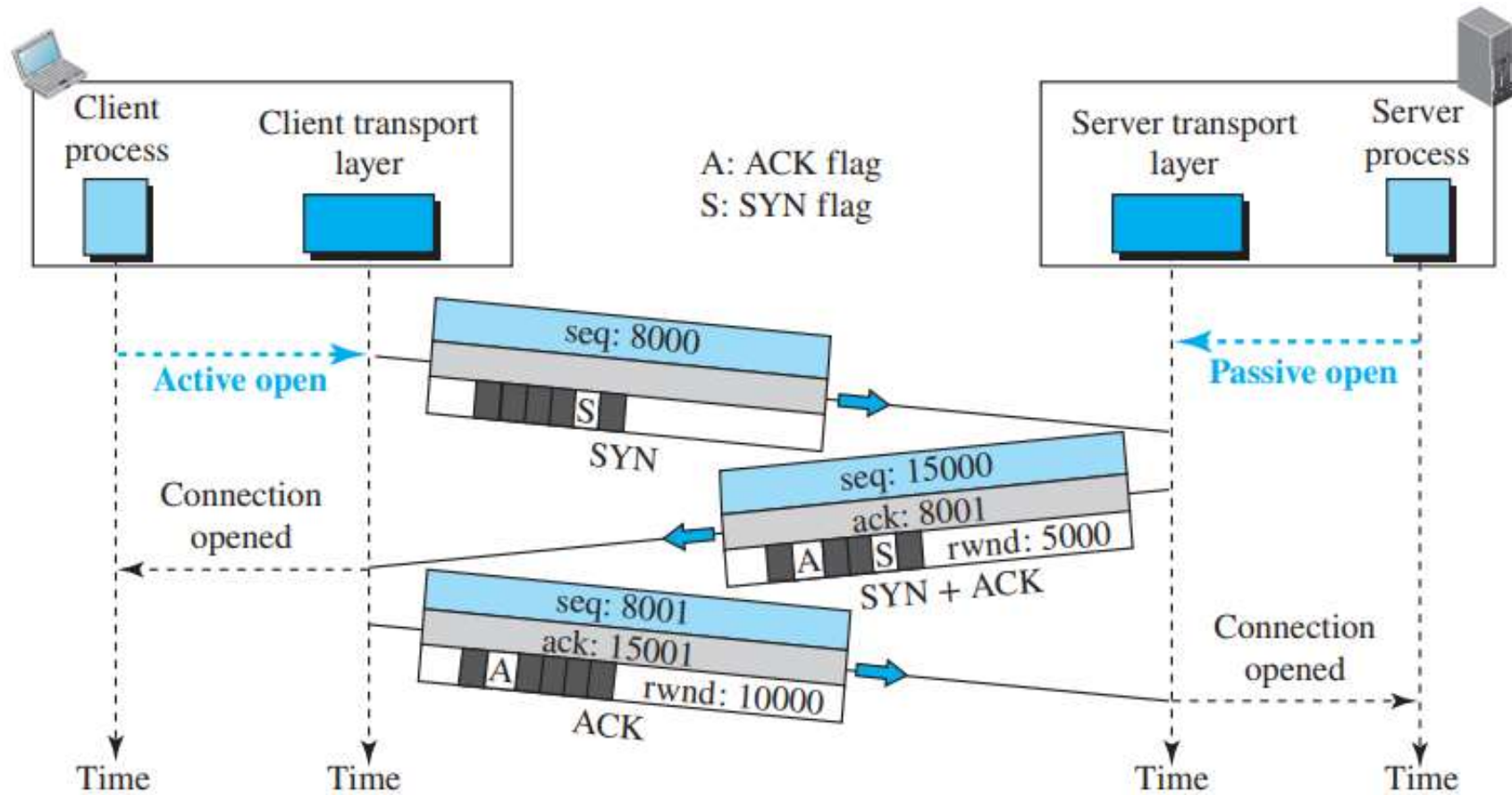
In TCP, connection-oriented transmission requires three phases:

- Connection establishment
- Data transfer
- Connection termination

Connection Establishment

- The connection establishment in TCP is called **Three-way Handshaking**.
- The process starts with the server.
 - The server program tells its TCP that it is ready to accept a connection.
 - This request is called a ***passive open***.
 - Although the server TCP is ready to accept a connection from any machine in the world, it cannot make the connection itself.
- The client program issues a request for an ***active open***.
 - A client that wishes to connect to an open server tells its TCP to connect to a particular server.
- TCP can now start the three-way handshaking process.

Connection establishment using three-way handshaking



1. The client sends the first segment, a SYN segment, in which only the SYN flag is set.

- This segment is for synchronization of sequence numbers.
- The client chooses a random number as the first sequence number and sends this number to the server.
- This sequence number is called the ***Initial Sequence Number (ISN)***.
- Note that this segment does not contain an acknowledgment number. It does not define the window size either; a window size definition makes sense only when a segment includes an acknowledgment.
- Note that the SYN segment is a control segment and carries no data.
- However, it consumes one sequence number because it needs to be acknowledged. Here, the SYN segment carries 1 imaginary byte.

A SYN segment cannot carry data, but it consumes one sequence number.

2. The server sends the second segment, a SYN + ACK segment with two flag bits set as SYN and ACK. This segment has a dual purpose.

- First, it is a SYN segment for communication in the other direction.
- The server uses this segment to initialize a sequence number for numbering the bytes sent from the server to the client.
- The server also acknowledges the receipt of the SYN segment from the client by setting the ACK flag and displaying the next sequence number it expects to receive from the client.
- Because it contains an acknowledgment, it also needs to define the receive window size, **rwnd** (to be used by the client).
- Because this segment is playing the role of a SYN segment, it needs to be acknowledged. It, therefore, consumes one sequence number.

**A SYN + ACK segment cannot carry data,
but it does consume one sequence number.**

3. The client sends the third segment. This is just an ACK segment.

- It acknowledges the receipt of the second segment with the ACK flag and acknowledgment number field.
- Note that the ACK segment does not consume any sequence numbers if it does not carry data, but some implementations allow this third segment in the connection phase to carry the first chunk of data from the client.
- In this case, the segment consumes as many sequence numbers as the number of data bytes.

If an ACK segment does not carry any data, it does not consume any sequence numbers.

SYN Flooding Attack

- The connection establishment procedure in TCP is susceptible to a serious security problem called a **SYN flooding attack**.
- This SYN flooding attack belongs to a group of security attacks known as a **Denial of Service attack**, in which an attacker monopolizes a system with so many service requests that the system overloads and denies service to valid requests.

Strategies to alleviate the effect of a SYN attack.

- Imposing a limit of connection requests during a specified period of time.
- Filter out datagrams coming from unwanted source addresses.
- Postpone resource allocation until the server can verify that the connection request is coming from a valid IP address, by using what is called a **Cookie**.

Data Transfer

- After a connection is established, bidirectional data transfer can take place.
- The client and server can send data and acknowledgments in both directions.
- The acknowledgment is piggybacked with the data.

Pushing Data

- The sending TCP uses a buffer to store the stream of data coming from the sending application program. The sending TCP can select the segment size.
- The receiving TCP also buffers the data when they arrive and delivers them to the application program when the application program is ready or when it is convenient for the receiving TCP.
- This type of flexibility increases the efficiency of TCP.

In some situations, delayed transmission and delayed delivery of data may not be acceptable by the application program.

- TCP can handle such a situation.
- The application program at the sender can request a push operation.
- This means that the sending TCP must not wait for the window to be filled. It must create a segment and send it immediately.
- The sending TCP must also set the **push bit (PSH)** to let the receiving TCP know that the segment includes data that must be delivered to the receiving application program as soon as possible and not to wait for more data to come.
- This means to change the *byte-oriented TCP* to a *chunk-oriented TCP*, but TCP can choose whether or not to use this feature.

Urgent Data

- TCP is a stream-oriented protocol. This means that the data are presented from the application program to TCP as a stream of bytes. Each byte of data has a position in the stream.
- However, there are occasions in which an application program needs to send **urgent bytes**, some bytes that need to be treated in a special way by the application at the other end.
- The **solution** is to send a segment with the **URG bit set**.
- The sending application program tells the sending TCP that the piece of data is urgent.
- The sending TCP creates a segment and inserts the urgent data at the beginning of the segment.
- The rest of the segment can contain normal data from the buffer.
- The urgent pointer field in the header defines the end of the urgent data (the last byte of urgent data).

For example, if the segment sequence number is 15,000 and the value of the urgent pointer is 200, the first byte of urgent data is the byte 15,000 and the last byte is the byte 15,200. The rest of the bytes in the segment (if present) are non-urgent.

Connection Termination

Either of the two parties involved in exchanging data (client or server) can close the connection, although it is usually initiated by the client.

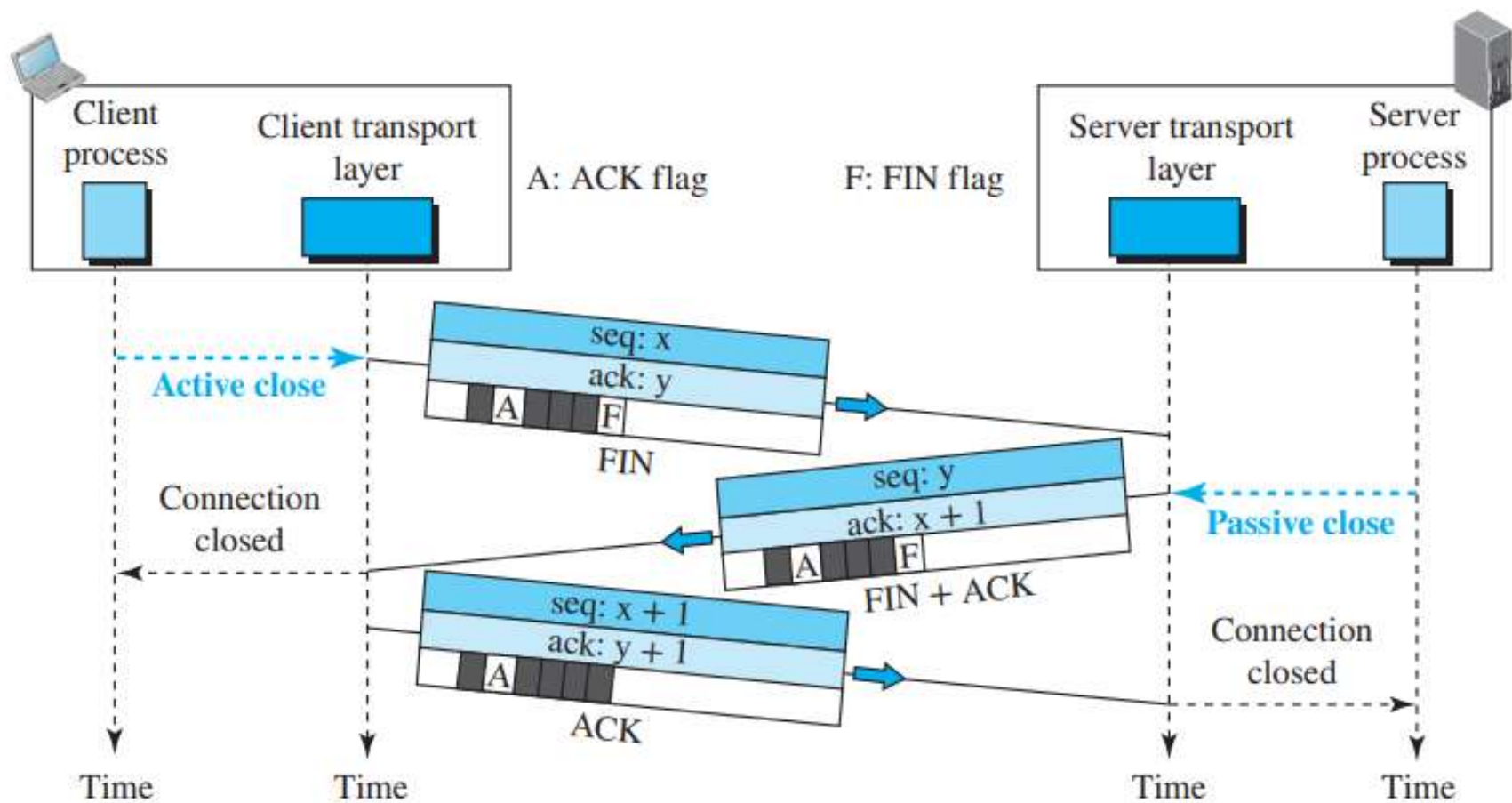
Most implementations today allow two options for connection termination:

- Three-way handshaking
- Four-way handshaking with a half-close option

Three-Way Handshaking

1. In this situation, the client TCP, after receiving a close command from the client process, sends the first segment, a FIN segment in which the FIN flag is set. Note that a FIN segment can include the last chunk of data sent by the client or it can be just a control segment. If it is only a control segment, it consumes only one sequence number because it needs to be acknowledged.
2. The server TCP, after receiving the FIN segment, informs its process of the situation and sends the second segment, a FIN + ACK segment, to confirm the receipt of the FIN segment from the client and at the same time to announce the closing of the connection in the other direction. This segment can also contain the last chunk of data from the server. If it does not carry data, it consumes only one sequence number because it needs to be acknowledged.
3. The client TCP sends the last segment, an ACK segment, to confirm the receipt of the FIN segment from the TCP server. This segment contains the acknowledgment number, which is one plus the sequence number received in the FIN segment from the server. This segment cannot carry data and consumes no sequence numbers.

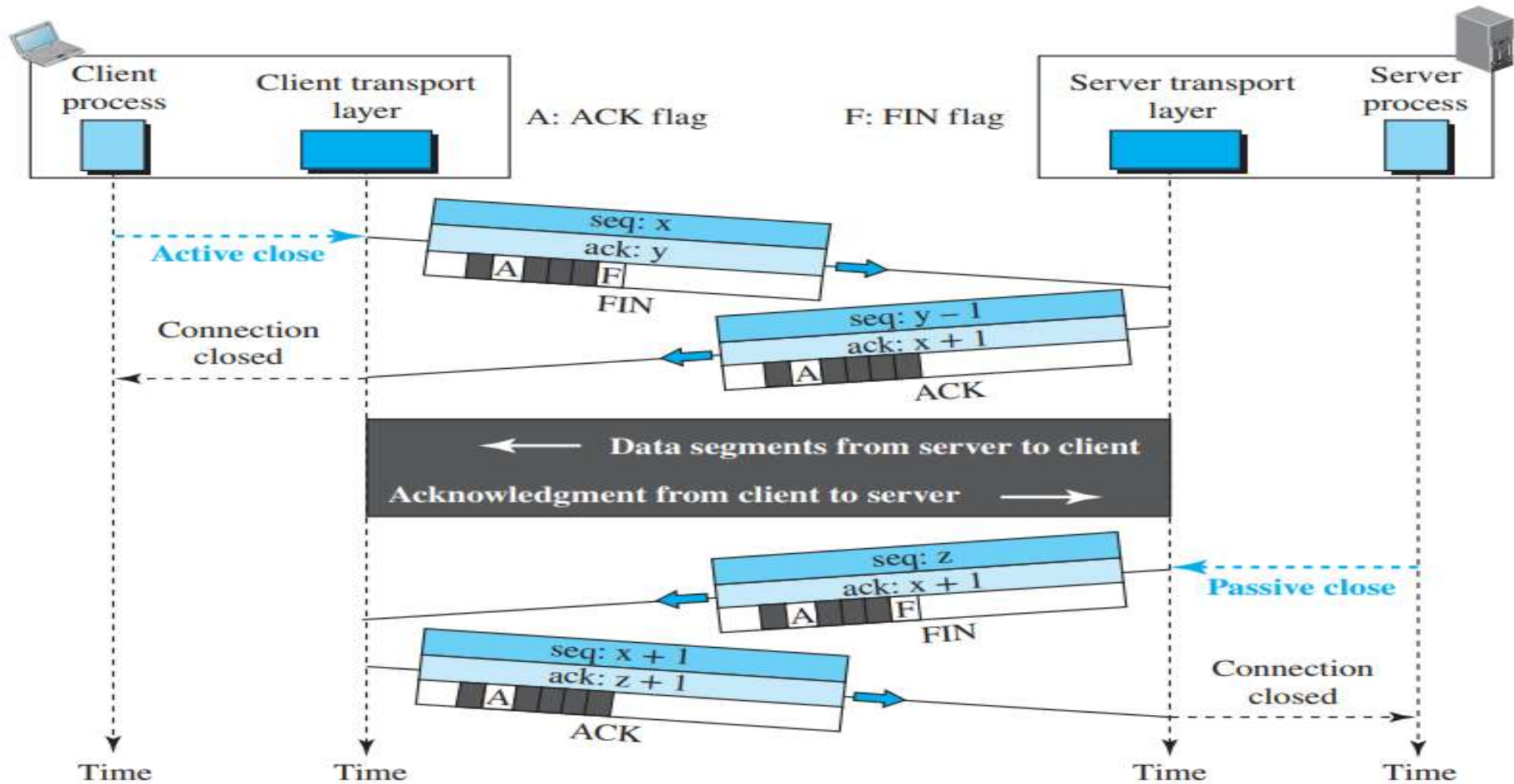
Connection termination using three-way handshaking



Half-Close

- In TCP, one end can stop sending data while still receiving data. This is called a **half-close**.
- Either the server or the client can issue a half-close request.
- It can occur when the server needs all the data before processing can begin.
- The data transfer from the client to the server stops.
- The client half-closes the connection by sending a **FIN** segment.
- The server accepts the half-close by sending the **ACK** segment. The server, however, can still send data.
- When the server has sent all the processed data, it sends a **FIN** segment, which is acknowledged by an **ACK** from the client.
- After half-closing the connection, data can travel from the server to the client and acknowledgments can travel from the client to the server. The client cannot send any more data to the server.

Half-close



Connection Reset

TCP at one end may deny a connection request, may abort an existing connection, or may terminate an idle connection. All these are done with the RST (reset) flag.

Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 10001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1000 bytes?

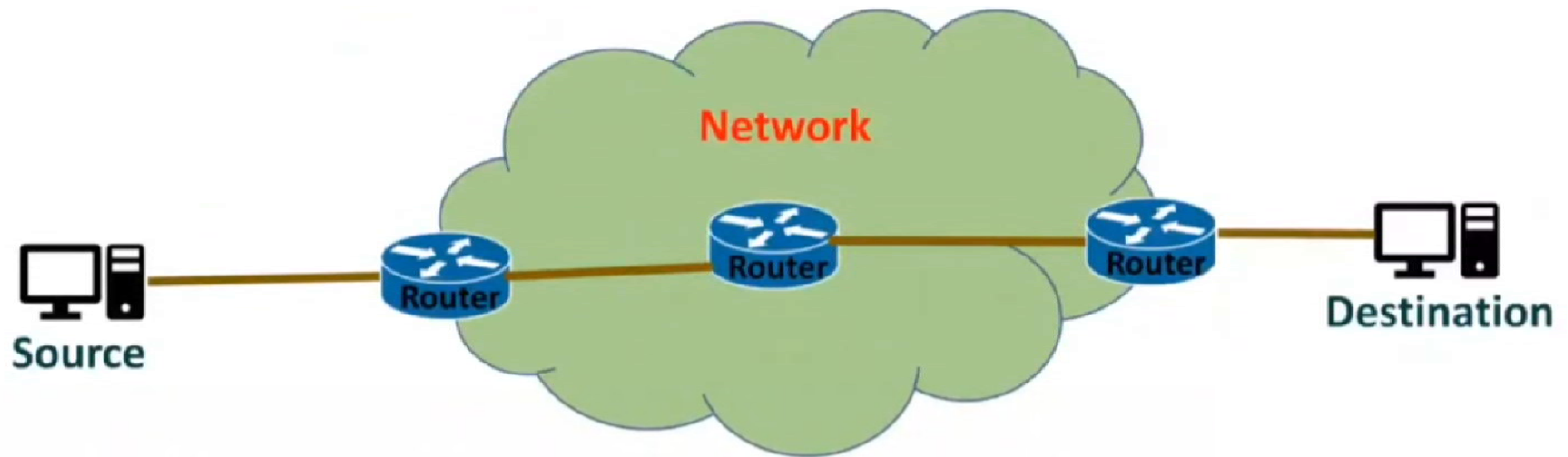
Solution

The following shows the sequence number for each segment:

Segment 1	→	Sequence Number:	10001	Range:	10001	to	11000
Segment 2	→	Sequence Number:	11001	Range:	11001	to	12000
Segment 3	→	Sequence Number:	12001	Range:	12001	to	13000
Segment 4	→	Sequence Number:	13001	Range:	13001	to	14000
Segment 5	→	Sequence Number:	14001	Range:	14001	to	15000

The value in the sequence number field of a segment defines the number assigned to the first data byte contained in that segment.

Congestion Control in TCP



Congestion control in Network layer ---- Between the source and the router. Handled by ICMP (**Internet Control Message Protocol**).

Congestion control in TCP layer ---- In the network

Congestion – Traffic in the network

Congestion detection:

- Time-out and 3 ACKs

Congestion control Policies:

- Slow start (Multiplicative increase or Exponential increase) --- used when heavy traffic
- Congestion Avoidance (Additive increase) --- used when light traffic
- Fast Recovery

Taho TCP and Reno TCP

Congestion Control Algorithm in TCP

□ Congestion control algorithm in the TCP has three stages:

1. Slow start phase

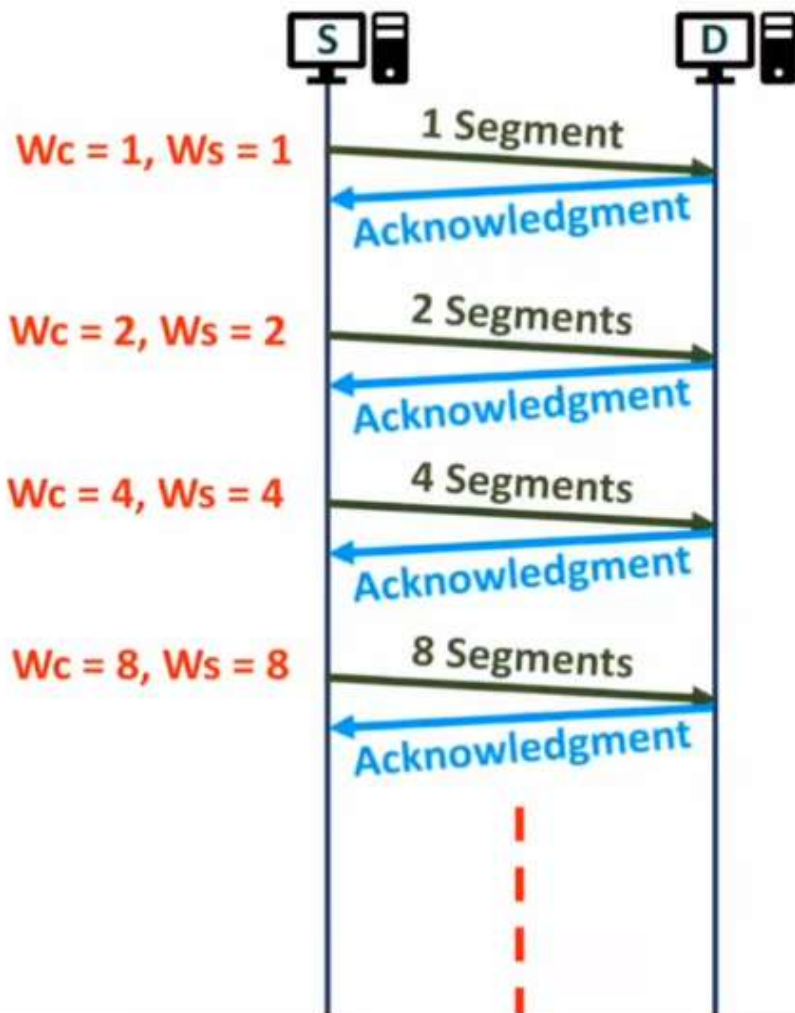
- Congestion window $Wc = 1$ or 2 initially.
- Wc will grow exponentially, it will get multiplied by 2 if there is no congestion up to the threshold value of the window.

2. Congestion Avoidance phase

- Wc will increase linearly by 1 if there is no congestion in the network.

3. Continuously Congestion detection phase

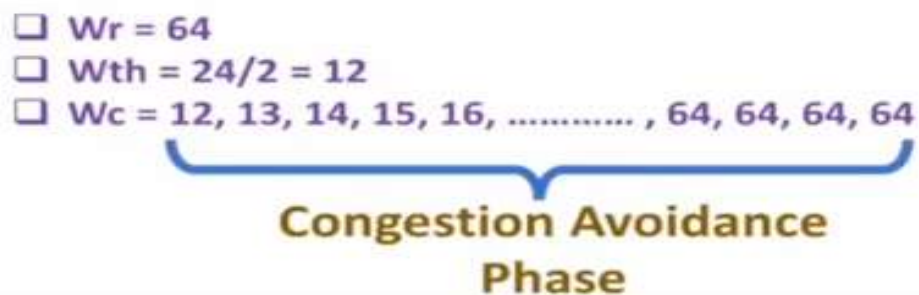
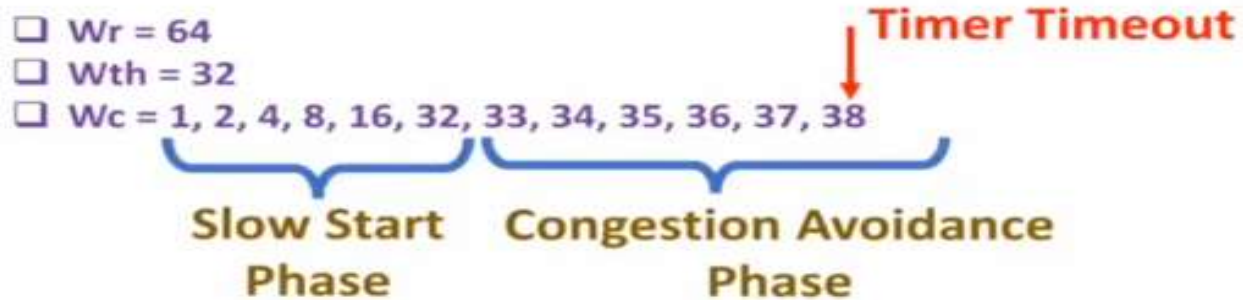
- Timeouts – Severe Congestion in the network.
 - $Wth = Wc / 2$ and slow start.
- 3 Dupack – Mild Congestion in the network.
 - $Wth = Wc / 2$ and congestion avoidance.

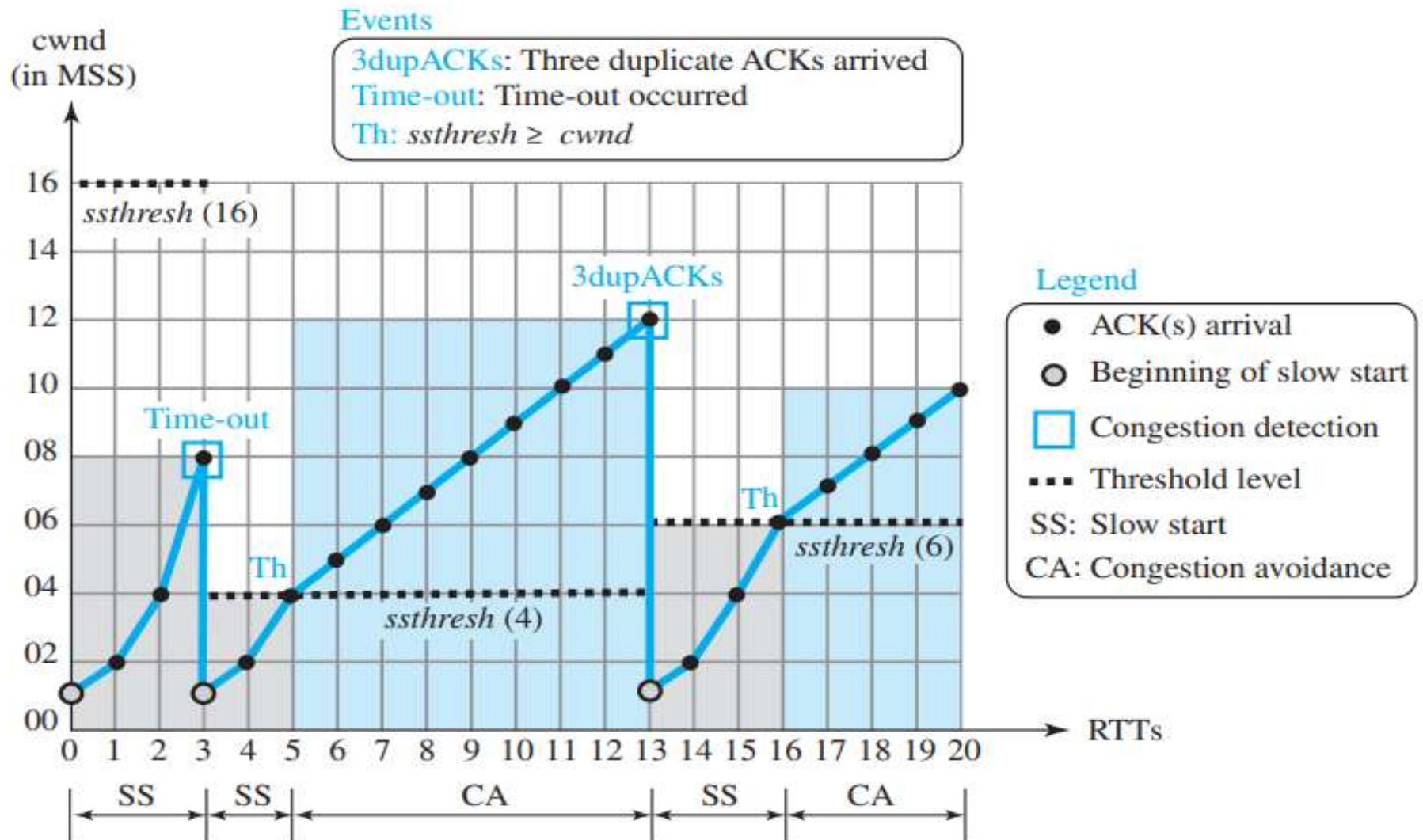


- ❑ Let's assume, $Ws = Wr = 64KB$ and $MSS = 1KB$.
- ❑ Here, for simplicity, I will consider, segment = 1KB
- ❑ Initially, $Wc = 1$ or 2, let's assume $Wc = 1$.
- ❑ $Ws = \min(Wr, Wc)$
- ❑ Initially, $Wth = Wr/2 = 32$

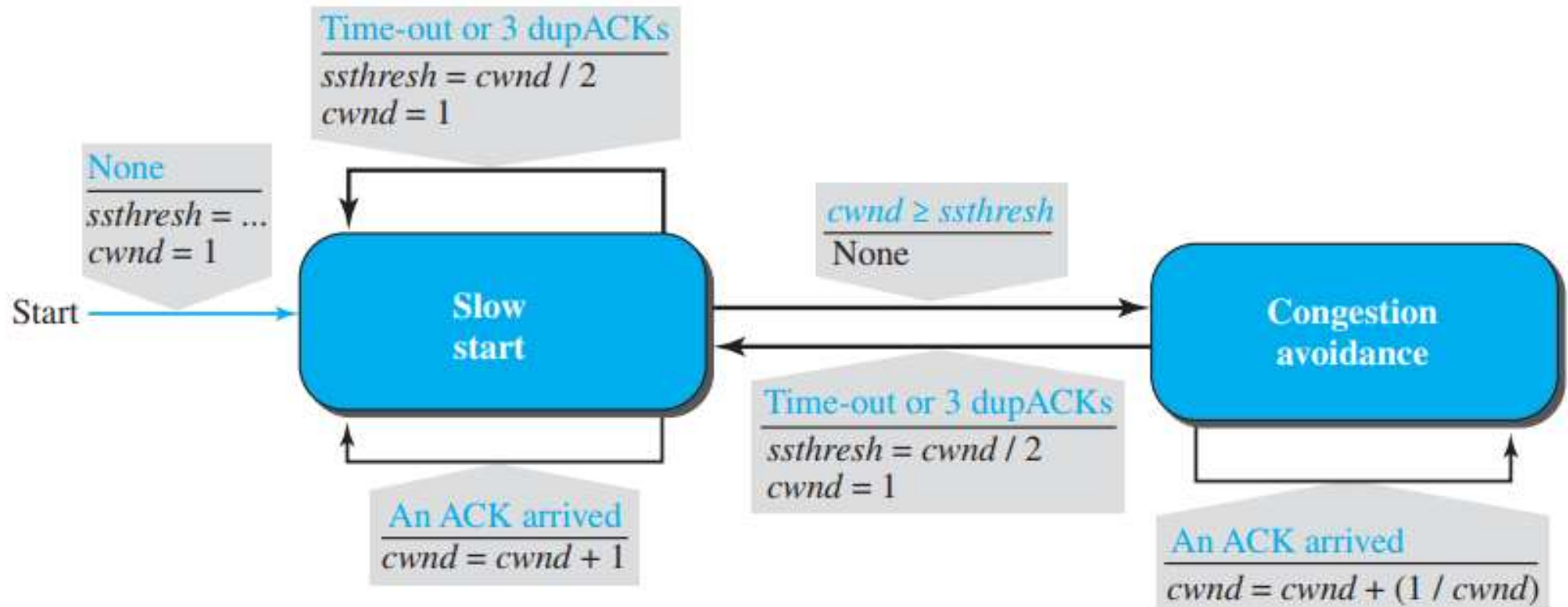


- ❑ Lets assume, $W_s = W_r = 64\text{KB}$ and $MSS = 1\text{KB}$.
- ❑ Here, for simplicity, I will consider, segment = 1KB
- ❑ Initially, $W_c = 1$ or 2, lest assume $W_c = 1$.
- ❑ $W_s = \min(W_r, W_c)$
- ❑ Initially, $W_{th} = W_r/2 = 32$

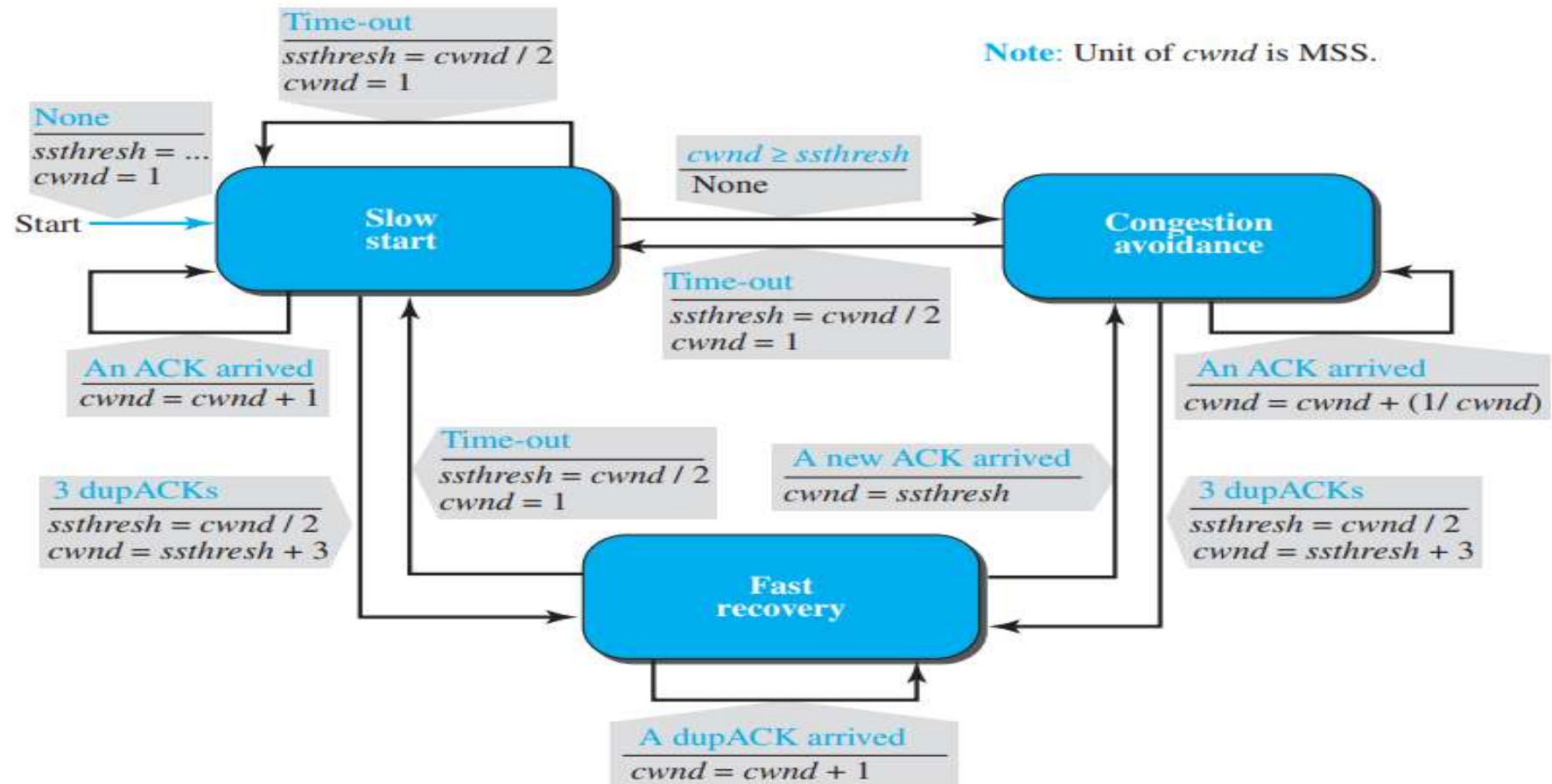




FSM for Tahoe TCP

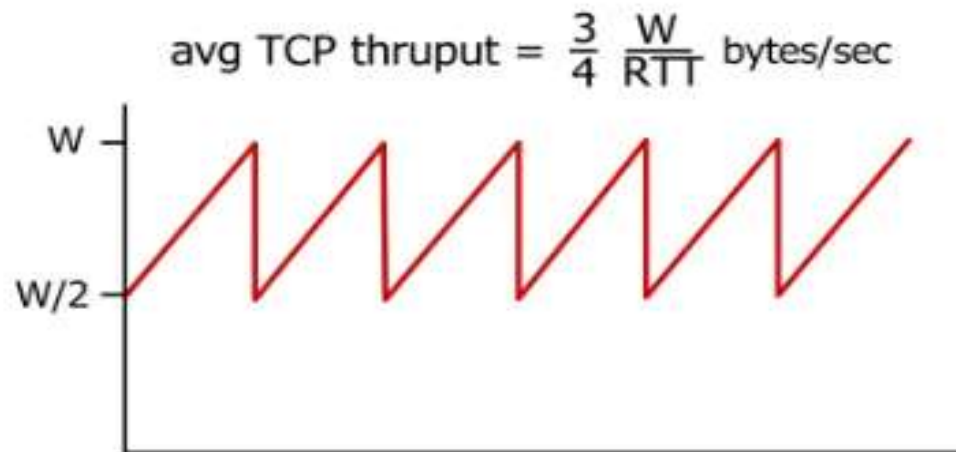


FSM for Reno TCP



TCP throughput

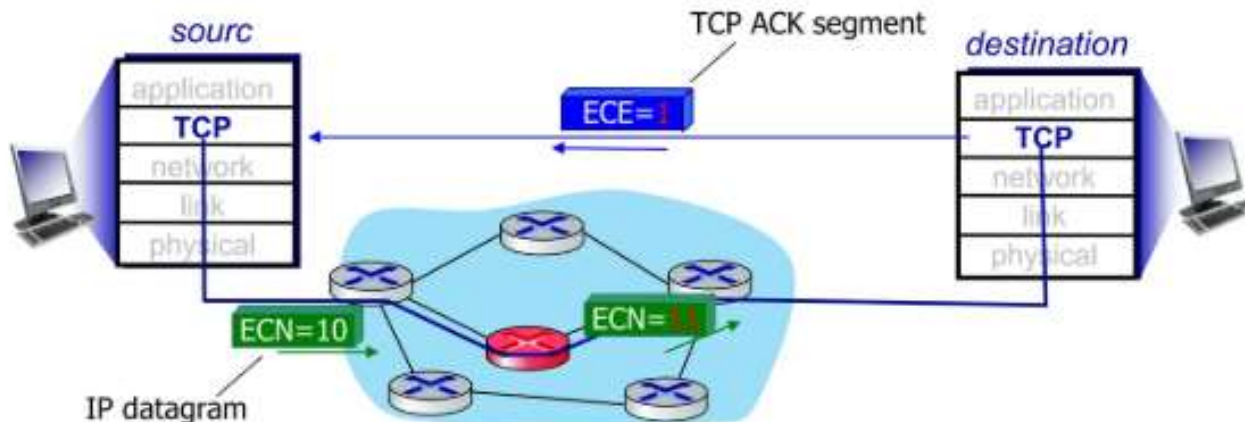
- avg. TCP throughput as function of window size, RTT?
 - ignore slow start, assume there is always data to send
- **W: window size** (measured in bytes) **where loss occurs**
 - avg. window size (# in-flight bytes) is $\frac{3}{4} W$
 - avg. thrupt is $\frac{3}{4}W$ per RTT



Explicit congestion notification (ECN)

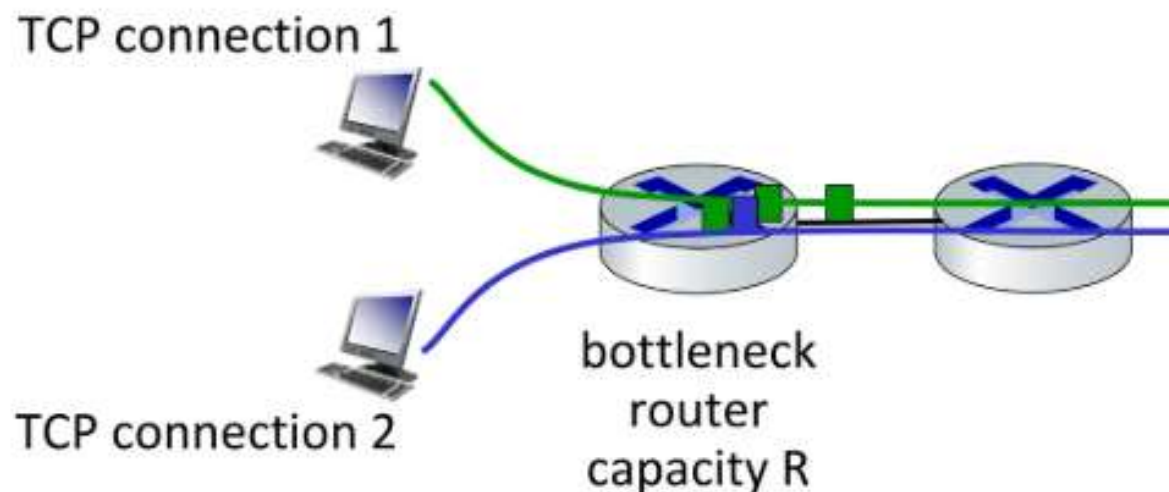
TCP deployments often implement *network-assisted* congestion control:

- two bits in IP header (ToS field) marked *by network router* to indicate congestion
 - *policy* to determine marking chosen by network operator
- congestion indication carried to destination
- destination sets ECE bit on ACK segment to notify sender of congestion
- involves both IP (IP header ECN bit marking) and TCP (TCP header C,E bit marking)



TCP fairness

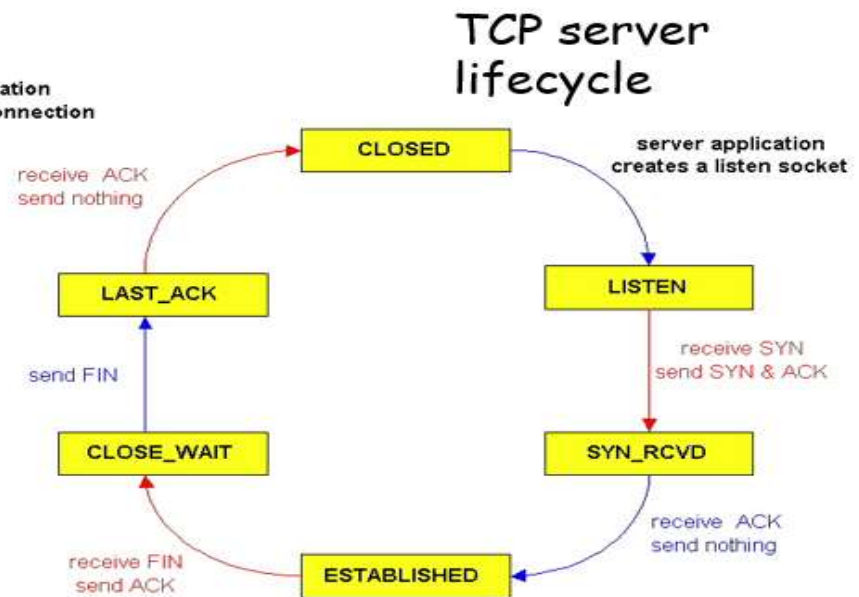
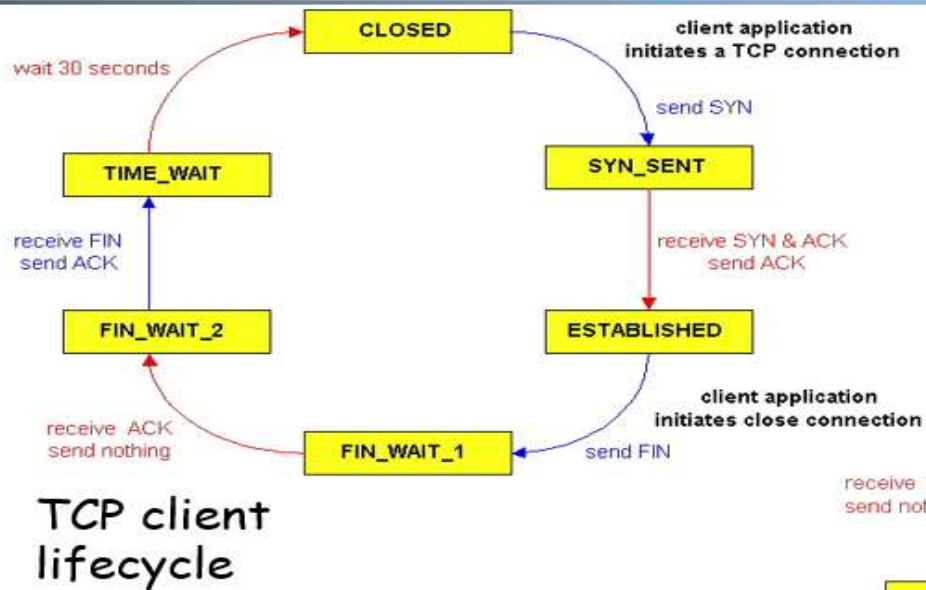
Fairness goal: if K TCP sessions share same bottleneck link of bandwidth R , each should have average rate of R/K



TCP sender congestion control

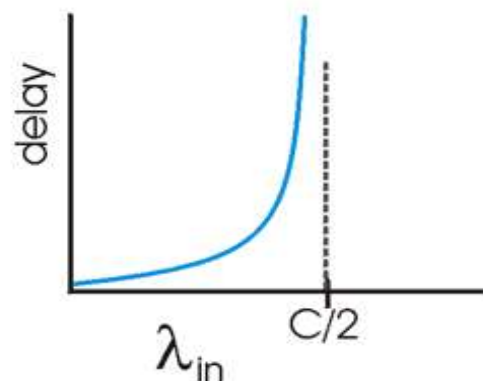
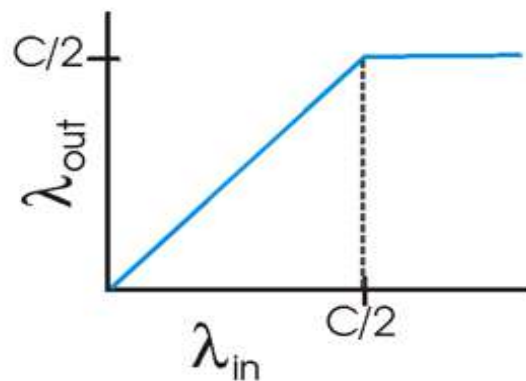
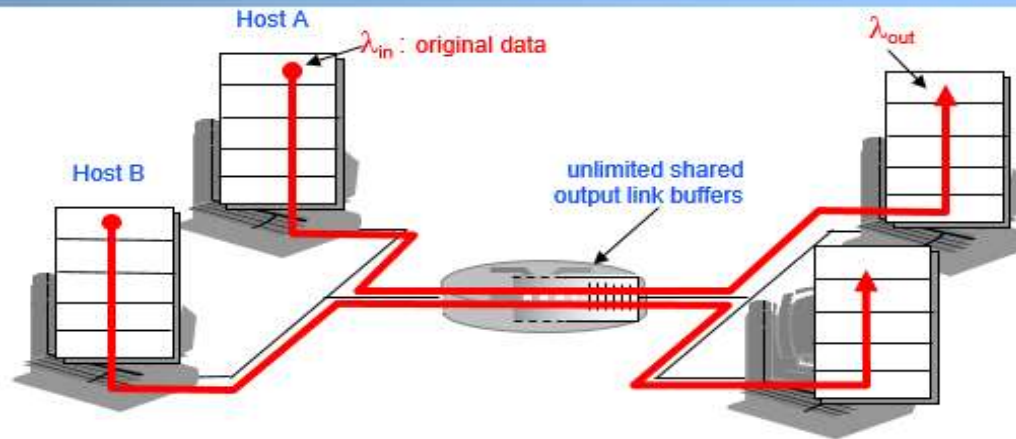
Event	State	TCP Sender Action	Commentary
ACK receipt for previously unacked data	Slow Start (SS)	$\text{CongWin} = \text{CongWin} + \text{MSS}$, If ($\text{CongWin} > \text{Threshold}$) set state to "Congestion Avoidance"	Resulting in a doubling of CongWin every RTT
ACK receipt for previously unacked data	Congestion Avoidance (CA)	$\text{CongWin} = \text{CongWin} + \text{MSS} * (\text{MSS} / \text{CongWin})$	Additive increase, resulting in increase of CongWin by 1 MSS every RTT
Loss event detected by triple duplicate ACK	SS or CA	$\text{Threshold} = \text{CongWin} / 2$, $\text{CongWin} = \text{Threshold}$, Set state to "Congestion Avoidance"	Fast recovery, implementing multiplicative decrease. CongWin will not drop below 1 MSS.
Timeout	SS or CA	$\text{Threshold} = \text{CongWin} / 2$, $\text{CongWin} = 1 \text{ MSS}$, Set state to "Slow Start"	Enter slow start
Duplicate ACK	SS or CA	Increment duplicate ACK count for segment being acked	CongWin and Threshold not changed

TCP Connection Management (cont)



Causes/costs of congestion: scenario 1

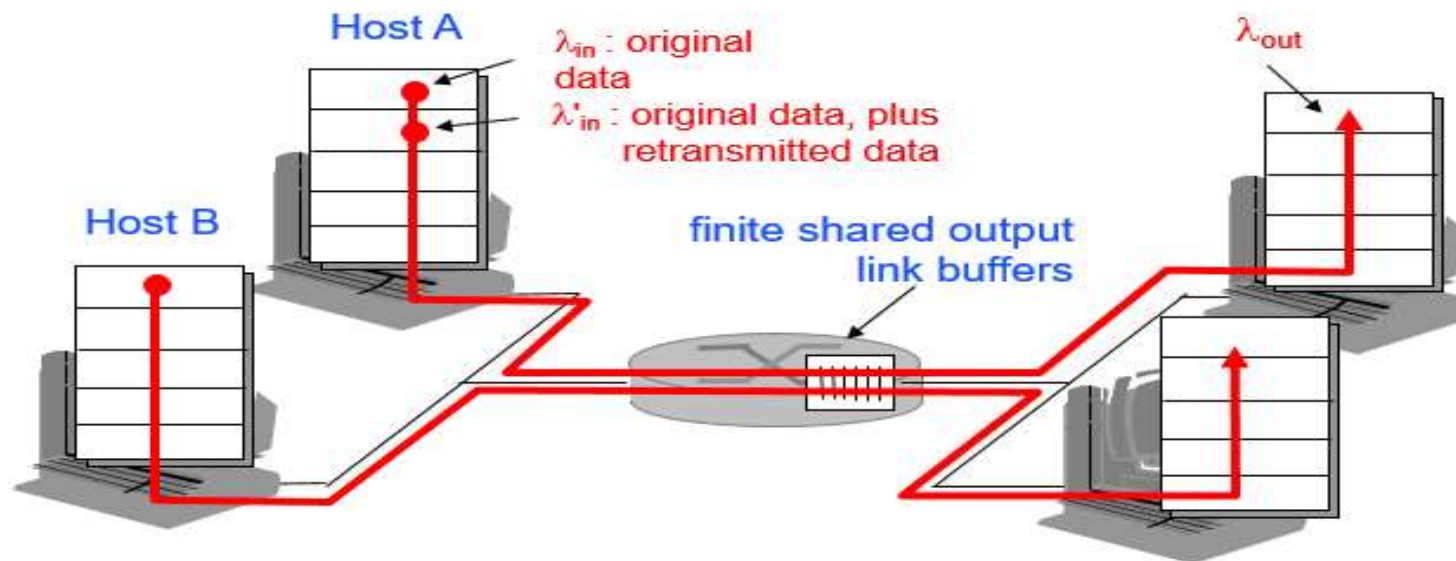
- two senders, two receivers
- one router, infinite buffers
- no retransmission



- large delays when congested
- maximum achievable throughput

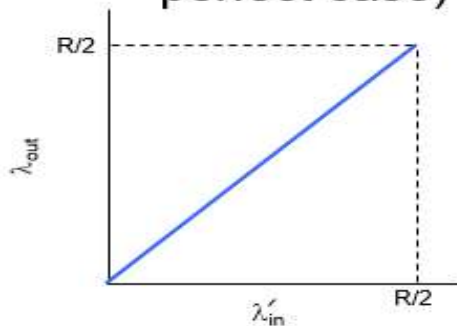
Causes/costs of congestion: scenario 2

- one router, *finite* buffers
- sender retransmission of lost packet

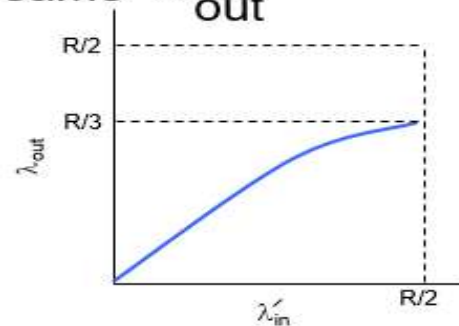


Causes/costs of congestion: scenario 2

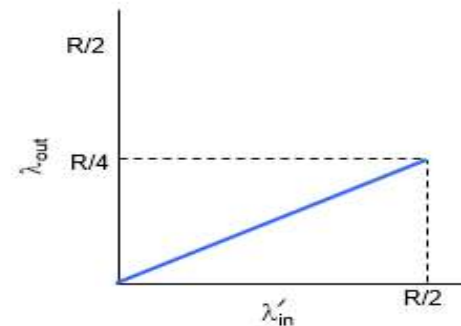
- always: $\lambda_{in} = \lambda_{out}$ (goodput)
- “perfect” retransmission only when loss: $\lambda'_{in} > \lambda_{out}$
- retransmission of delayed (not lost) packet makes λ'_{in} larger (than perfect case) for same λ_{out}



a.



b.



c.

“costs” of congestion:

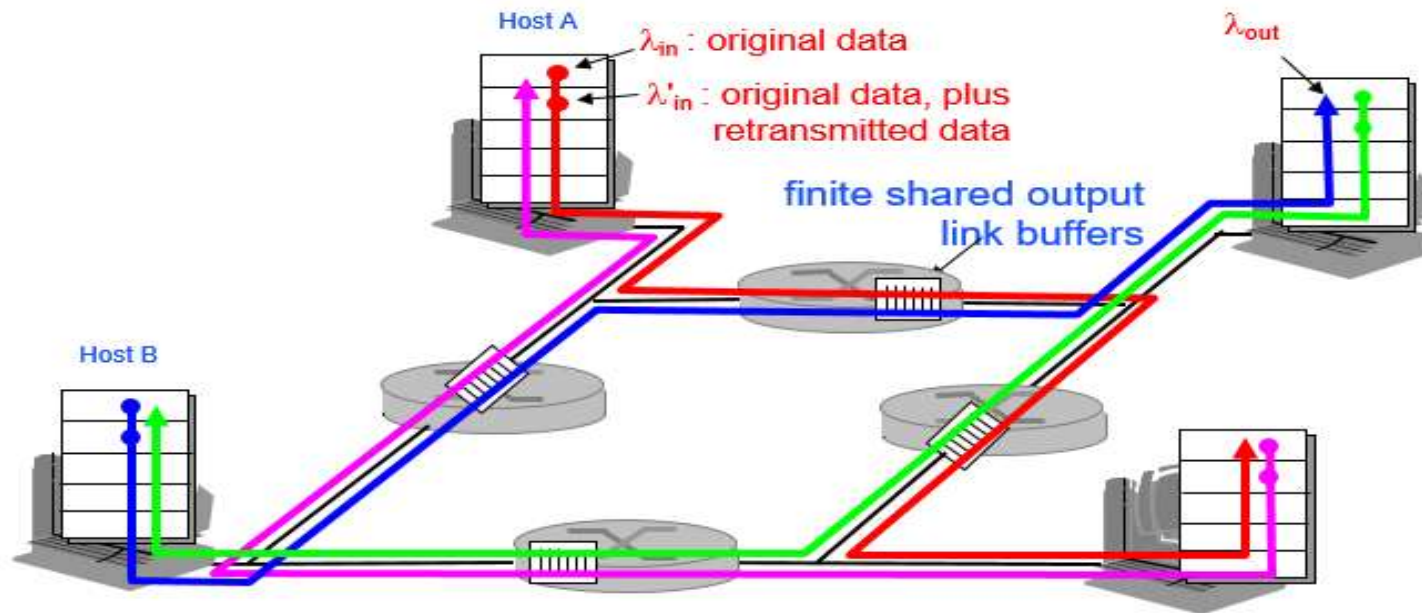
more work (retrans) for given “goodput”

unnneeded retransmissions: link carries multiple copies of pkt ..

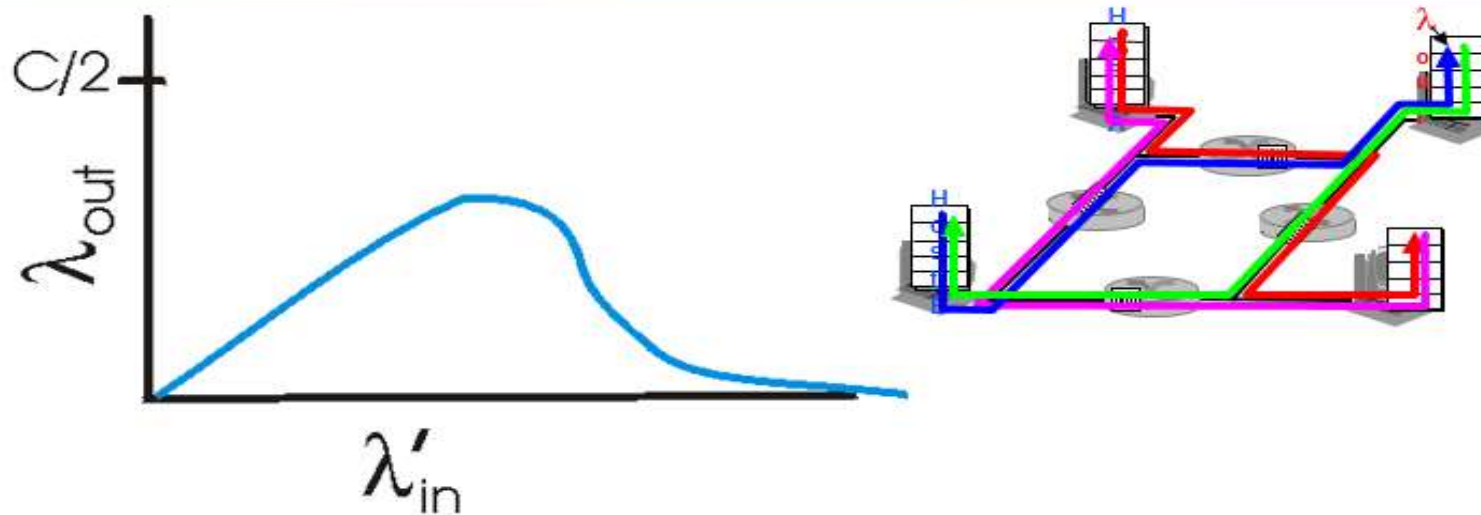
Causes/costs of congestion: scenario 3

- four senders
- multihop paths
- timeout/retransmit

Q: what happens as λ_{in} and λ'_{in} increase ?



Causes/costs of congestion: scenario 3



Another “cost” of congestion:

when packet dropped, any “upstream transmission capacity used for that packet was wasted!

Approaches towards congestion control

Two broad approaches towards congestion control:

End-end congestion control:

- no explicit feedback from network
- congestion inferred from end-system observed loss, delay
- approach taken by TCP

Network-assisted congestion control:

- routers provide feedback to end systems
 - single bit indicating congestion (SNA, DECbit, TCP/IP ECN, ATM)
 - explicit rate sender should send at

TCP throughput

- What's the average throughput of TCP as a function of window size and RTT?
 - Ignore slow start
- Let W be the window size when loss occurs.
- When window is W , throughput is W/RTT
- Just after loss, window drops to $W/2$, throughput to $W/2RTT$.
- Average throughput: $.75 W/RTT$

TCP Futures

- Example: 1500 byte segments, 100ms RTT, want 10 Gbps throughput
- Requires window size $W = 83,333$ in-flight segments
- Throughput in terms of loss rate:

$$\frac{1.22 \cdot MSS}{RTT \sqrt{L}}$$

- $\rightarrow L = 2 \cdot 10^{-10}$ **Wow**
- New versions of TCP for high-speed needed!

Delay modeling

Q: How long does it take to receive an object from a Web server after sending a request?

Ignoring congestion, delay is influenced by:

- TCP connection establishment
- data transmission delay
- slow start

Notation, assumptions:

- Assume one link between client and server of rate R
- S : MSS (bits)
- O : object size (bits)
- no retransmissions (no loss, no corruption)

Window size:

- First assume: fixed congestion window, W segments
- Then dynamic window, modeling slow start

TCP Delay Modeling: Slow Start (1)

Now suppose window grows according to slow start

Will show that the delay for one object is:

$$Latency = 2RTT + \frac{O}{R} + P \left[RTT + \frac{S}{R} \right] - (2^P - 1) \frac{S}{R}$$

where P is the number of times TCP idles at server:

$$P = \min\{Q, K - 1\}$$

- where Q is the number of times the server idles if the object were of infinite size.
- and K is the number of windows that cover the object.

TCP Delay Modeling: Slow Start (2)

Delay components:

- 2 RTT for connection estab and request
- O/R to transmit object
- time server idles due to slow start

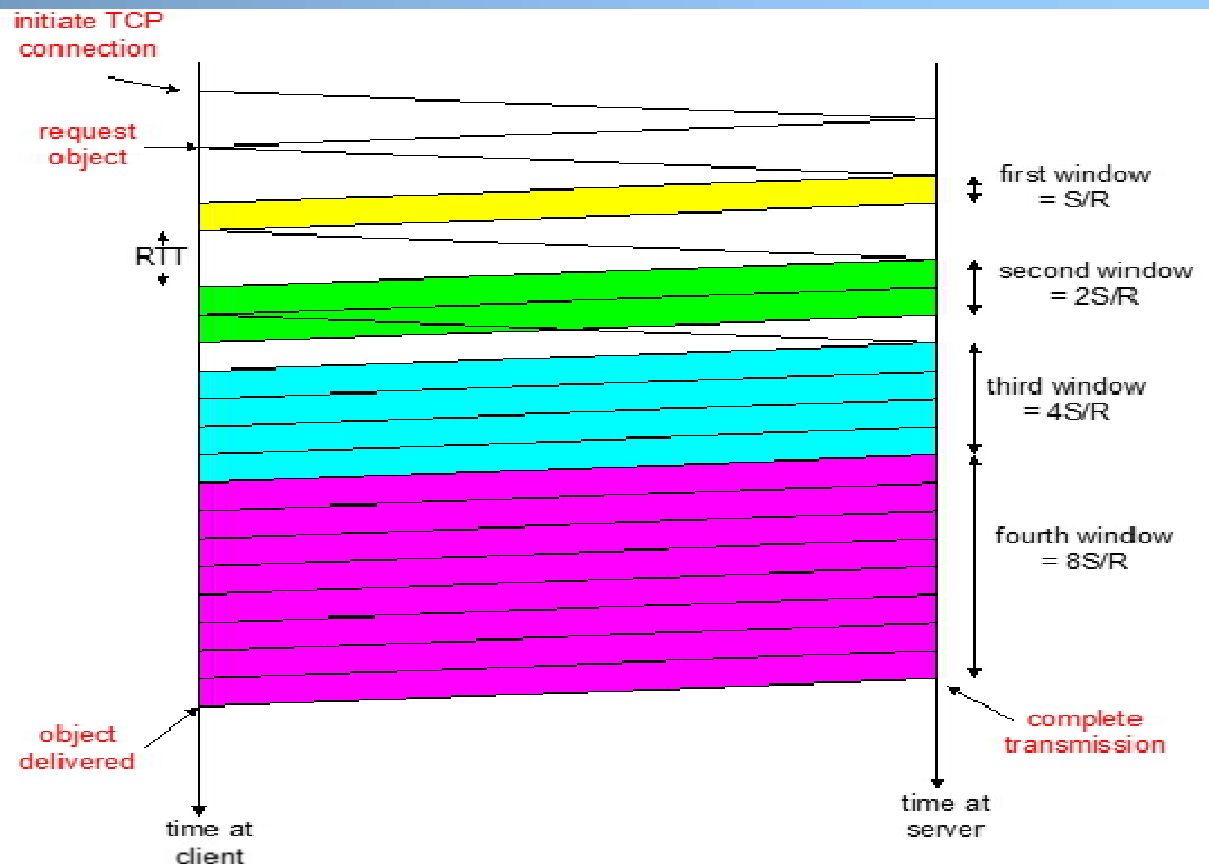
Server idles:

$$P = \min\{K-1, Q\} \text{ times}$$

Example:

- $O/S = 15$ segments
- $K = 4$ windows
- $Q = 2$
- $P = \min\{K-1, Q\} = 2$

Server idles $P=2$ times



TCP Delay Modeling (3)

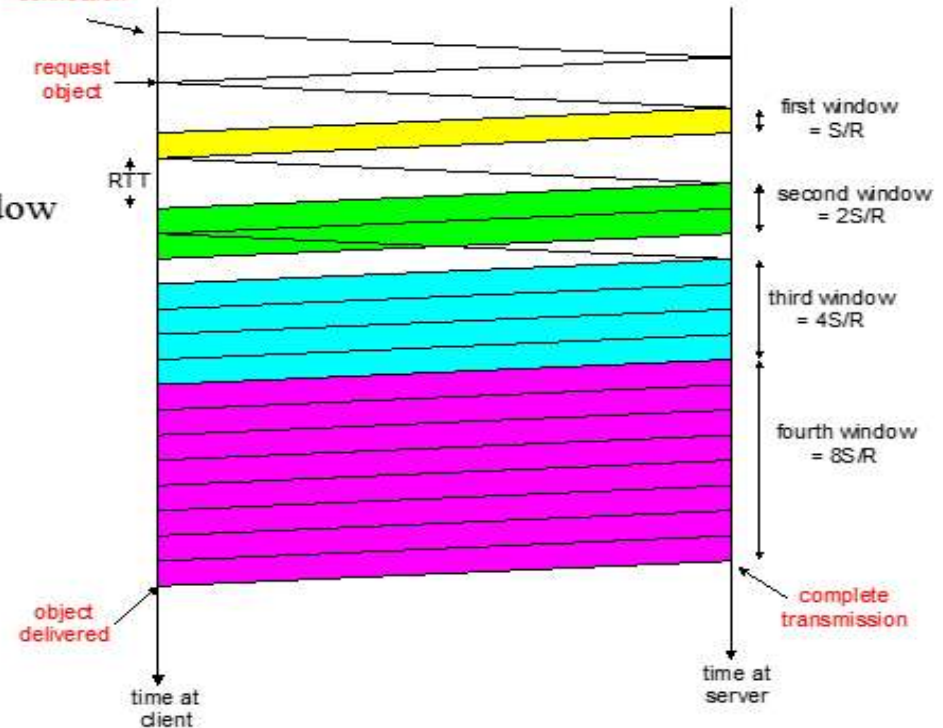
$\frac{S}{R} + RTT$ = time from when server starts to send segment

until server receives acknowledgement

$2^{k-1} \frac{S}{R}$ = time to transmit the k th window

$\left[\frac{S}{R} + RTT - 2^{k-1} \frac{S}{R} \right]^+$ = idle time after the k th window

$$\begin{aligned} \text{delay} &= \frac{O}{R} + 2RTT + \sum_{p=1}^P \text{idleTime}_p \\ &= \frac{O}{R} + 2RTT + \sum_{k=1}^P \left[\frac{S}{R} + RTT - 2^{k-1} \frac{S}{R} \right] \\ &= \frac{O}{R} + 2RTT + P \left[RTT + \frac{S}{R} \right] - (2^P - 1) \frac{S}{R} \end{aligned}$$

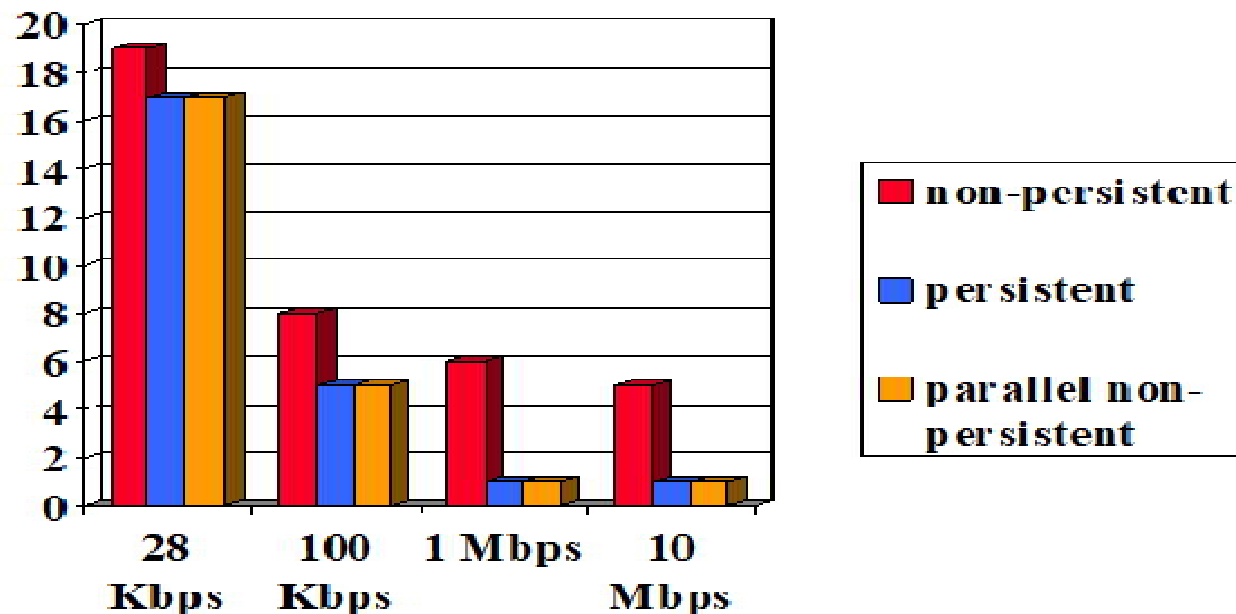


HTTP Modeling

- Assume Web page consists of:
 - 1 base HTML page (of size O bits)
 - M images (each of size O bits)
- Non-persistent HTTP:
 - $M+1$ TCP connections in series
 - *Response time = $(M+1)O/R + (M+1)2RTT + \text{sum of idle times}$*
- Persistent HTTP:
 - 2 RTT to request and receive base HTML file
 - 1 RTT to request and receive M images
 - *Response time = $(M+1)O/R + 3RTT + \text{sum of idle times}$*
- Non-persistent HTTP with X parallel connections
 - Suppose M/X integer.
 - 1 TCP connection for base file
 - M/X sets of parallel connections for images.
 - *Response time = $(M+1)O/R + (M/X + 1)2RTT + \text{sum of idle times}$*

HTTP Response time (in seconds)

RTT = 100 msec, O = 5 Kbytes, M=10 and X=5

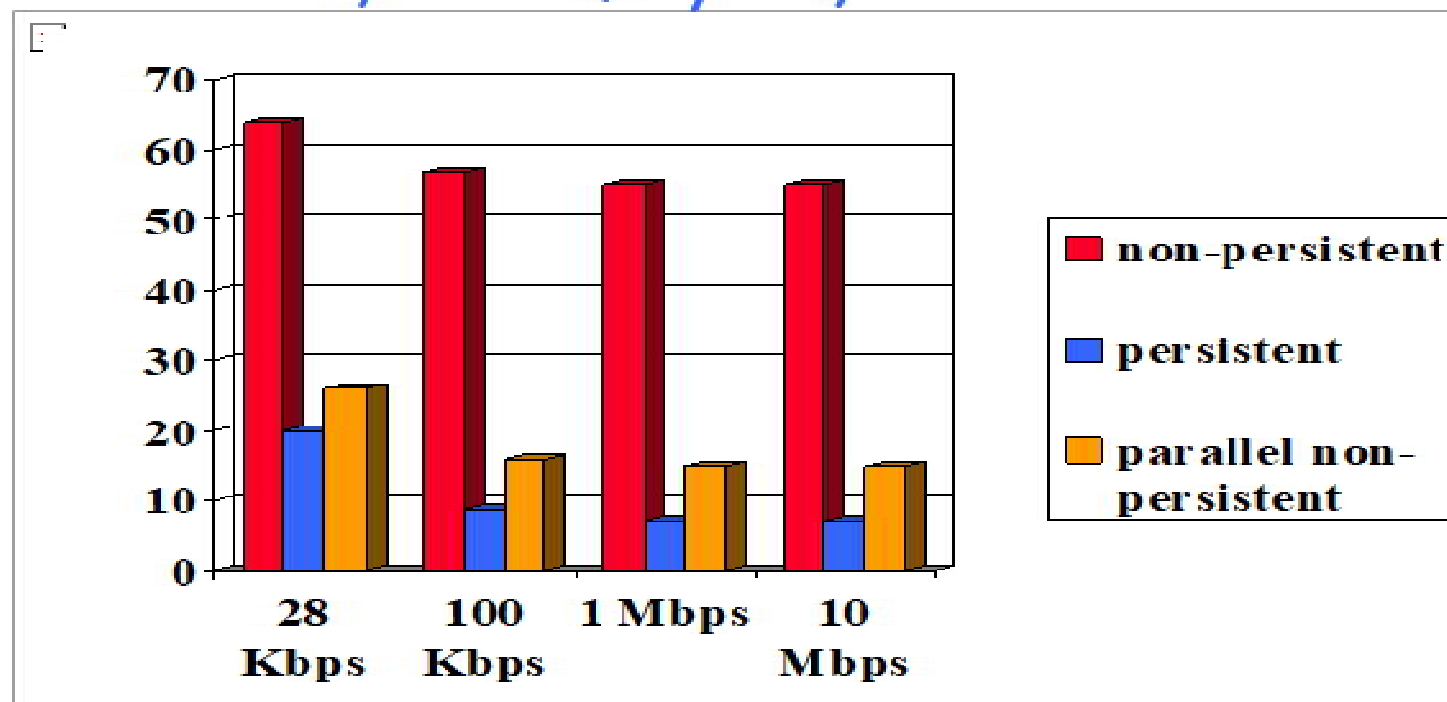


For low bandwidth, connection & response time dominated by transmission time.

Persistent connections only give minor improvement over parallel connections.

HTTP Response time (in seconds)

RTT = 1 sec, O = 5 Kbytes, M=10 and X=5



For larger RTT, response time dominated by TCP establishment & slow start delays. Persistent connections now give important improvement: particularly in high delay•bandwidth networks.

TCP Congestion Control

- TCP has a mechanism for congestion control. The mechanism is implemented at the sender
- The window size at the sender is set as follows:

Send Window = MIN (flow control window, congestion window)

where

- **flow control window** is advertised by the receiver
- **congestion window** is adjusted based on feedback from the network

TCP Congestion Control

- TCP congestion control is governed by two parameters:
 - **Congestion Window (cwnd)**
 - **Slow-start threshold Value (ssthresh)**
Initial value is $2^{16}-1$
- Congestion control works in two modes:
 - **slow start** ($cwnd < ssthresh$)
 - **congestion avoidance** ($cwnd \geq ssthresh$)

Slow Start

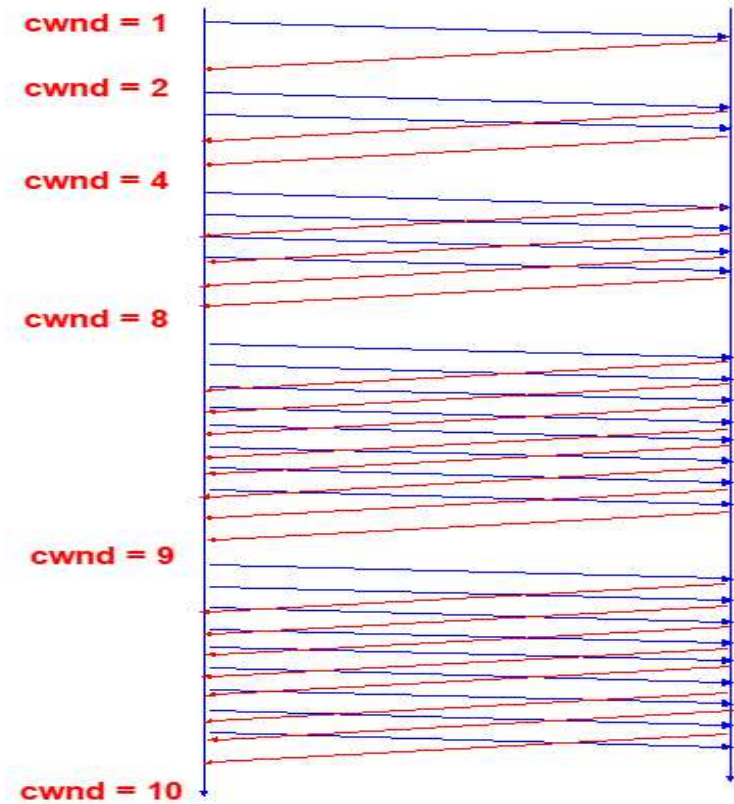
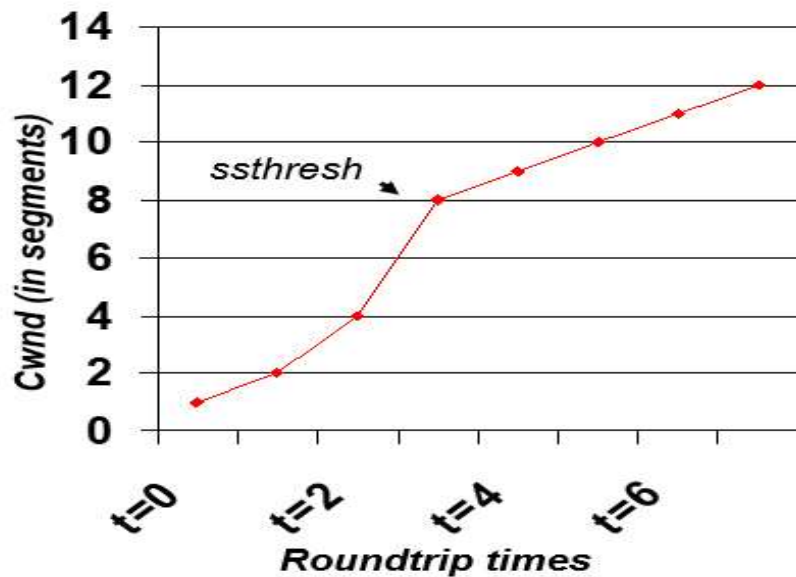
- Initial value: **Set cwnd = 1**
 - » Note: Unit is a segment size. TCP actually is based on bytes and increments by 1 MSS (maximum segment size)
- The receiver sends an acknowledgement (ACK) for each Segment
 - » Note: Generally, a TCP receiver sends an ACK for every other segment.
- Each time an ACK is received by the sender, the congestion window is increased by 1 segment:
cwnd = cwnd + 1
 - » If an ACK acknowledges two segments, cwnd is still increased by only 1 segment.
 - » Even if ACK acknowledges a segment that is smaller than MSS bytes long, cwnd is increased by 1.
- Does Slow Start increment slowly? Not really.
In fact, the increase of cwnd is exponential

Congestion Avoidance

- Congestion avoidance phase is started if *cwnd* has reached the slow-start threshold value
- If $cwnd \geq ssthresh$ then each time an ACK is received, increment *cwnd* as follows:
 - $cwnd = cwnd + 1 / cwnd$
- So *cwnd* is increased by one only if all *cwnd* segments have been acknowledged.

Example of Slow Start/Congestion Avoidance

Assume that *ssthresh* = 8



Responses to Congestion

- So, TCP assumes there is congestion if it detects a packet loss
- A TCP sender can detect lost packets via:
 - Timeout of a retransmission timer
 - Receipt of a duplicate ACK
- TCP interprets a Timeout as a binary congestion signal. When a timeout occurs, the sender performs:
 - cwnd is reset to one:
 $\text{cwnd} = 1$
 - ssthresh is set to half the current size of the congestion window:
 $\text{ssthresh} = \text{cwnd} / 2$
 - and slow-start is entered

Summary of TCP congestion control

Initially:

```
  cwnd = 1;  
  ssthresh =  
    advertised window size;
```

New Ack received:

```
  if (cwnd < ssthresh)  
    /* Slow Start */  
    cwnd = cwnd + 1;  
  else  
    /* Congestion Avoidance */  
    cwnd = cwnd + 1/cwnd;
```

Timeout:

```
  /* Multiplicative decrease */  
  ssthresh = cwnd/2;  
  cwnd = 1;
```

Flavors of TCP Congestion Control

- **TCP Tahoe** (1988, FreeBSD 4.3 Tahoe)
 - Slow Start
 - Congestion Avoidance
 - Fast Retransmit
 - **TCP Reno** (1990, FreeBSD 4.3 Reno)
 - Fast Recovery
 - **New Reno** (1996)
 - **SACK** (1996)
-
- **RED** (Floyd and Jacobson 1993)

TCP Tahoe and TCP Reno

(for single segment losses)

